

Bachelor's Thesis

Interpretable Machine Learning Models for Predicting Credit Approval

Department of Statistics
Ludwig-Maximilians-Universität München

Daniel Müller

Munich, July 20th, 2025



Submitted in partial fulfillment of the requirements for the degree of B. Sc.
Supervised by Dr. Ludwig Bothmann

Abstract

This paper analyzes and compares the performance and interpretability of interpretable and non-interpretable models for predicting credit approval using Home Mortgage Disclosure Act data from 2023 for the District of Columbia. Among the interpretable models, logistic regression and a classification tree are considered, and among the non-interpretable models, a random forest and two support vector machines are investigated. To this end, various evaluation metrics, feature importance methods, and partial dependence plots are applied to each model. As the study progresses, it becomes evident that the interpretable models exhibit performance similar to that of the non-interpretable models. While the results of the feature importance methods and partial dependence plots largely agree across all models, some notable differences also emerge. In conclusion, interpretable models present a competitive alternative to non-interpretable models for predicting credit approval, particularly in contexts where model transparency is required.

Contents

1	Introduction	1
2	Related Work	2
3	Theoretical Background	3
3.1	Interpretable Machine Learning	3
3.2	Evaluation Metrics	4
3.3	Feature Importance Methods	6
3.4	Feature Effect Methods	8
3.5	Empirical Risk Minimization	9
4	Dataset	10
5	Interpretable Models	11
5.1	Linear Model	11
5.2	Classification Tree	15
6	Non-Interpretable Models	20
6.1	Random Forest	20
6.2	Support Vector Machine	24
7	Interpretable vs. Non-Interpretable Models	31
7.1	Performance	31
7.2	Feature Importance	32
7.3	Feature Effect	34
8	Rashomon Effect	37
9	Discussion	38
10	Conclusion	39
A	Appendix	V
B	Electronic appendix	VII
C	Informations on used AI Tools	VIII

1 Introduction

The evaluation of creditworthiness plays a central role in the financial sector. On the one hand, interest on loans is a significant source of revenue for banks; on the other hand, loans pose a risk to banks if they cannot be repaid. Accordingly, the decision to grant a loan is a critical task for credit institutions. As automation continues to advance across various sectors, financial institutions are also increasingly relying on automated systems to make such credit decisions. Unsurprisingly, machine learning models are being applied more and more for this purpose. In recent years, a variety of complex, difficult-to-understand black box models have been introduced to determine credit approval or denial. These include models such as neural networks (see Guotai et al., 2017), support vector machines (see Chen and Li, 2014), and hybrid models (see Weng et al., 2021). However, applying such complex models comes with challenges related to transparency, interpretability, and regulatory requirements, including the issue of fairness and legal mandates (see Leal, 2024). Hence, the use of more interpretable models represents a compelling alternative in this domain.

The objective of this bachelor's thesis is to investigate and analyze interpretable machine learning models (e.g., classification trees and linear models) for predicting creditworthiness using an Home Mortgage Disclosure Act (HMDA) dataset (see Section 4). In addition, the thesis discusses the strengths of these models and assesses how they can be effectively employed as viable alternatives to opaque black box models.

First, the theoretical background relevant for the subsequent analysis is presented, including the evaluation metrics and the feature importance methods used (see Section 3).

Subsequently, two different interpretable models (i.e., a linear model and a classification tree) are trained and analyzed (see Section 5).

Following this, three non-interpretable models (i.e., a random forest and two support vector machines) are trained (see Section 6).

These models are then compared in various ways, including overall performance, feature importance, and feature effects (see Section 7).

Finally, the so-called Rashomon Effect is discussed and examined, analyzing how the interpretation can differ for different models with similar performance (see Section 8).

2 Related Work

In recent decades, significant progress has been made in the field of machine learning, resulting in the development of increasingly sophisticated and advanced models. Among these are complex, generally non-interpretable models, such as neural networks.

This raises the question of how such complex, non-interpretable models differ from simpler, interpretable ones, and to what extent they do so. Loyola-González (2019) examined the strengths and weaknesses of both model types. They found that, in many areas, complex, non-interpretable models deliver better results, while interpretable models provide understandable explanations for their predictions. However, they also emphasized that interpretable models can perform just as well as non-interpretable models. Chakraborty et al. (2021) examined interpretable and non-interpretable models for modeling hydro-climatological processes, while Zhao et al. (2023) investigated both model types for estimating winter wheat evapotranspiration. Both studies concluded that interpretable models can achieve results comparable to those of non-interpretable models.

In the context of credit scoring, several studies have also used data from the HMDA database. For example, Pillai et al. (2020) used data from 2009 to 2017, whereas Hodges et al. (2024) analyzed data from 2007 to 2017. Both studies trained various models to predict credit approval and compared their results. Zhang (2022) examined the fairness of models on HMDA data from 2019 with respect to applicant ethnicity and race.

In this paper, we compare interpretable and non-interpretable models using HMDA data from 2023.

3 Theoretical Background

3.1 Interpretable Machine Learning

In recent years, machine learning has advanced significantly, particularly through the development of complex models, such as deep neural networks. While these models generally provide highly accurate predictions, their internal structures make their inner mechanisms difficult to understand. As a result, their decision-making processes are hard or impossible for humans to understand. To address this challenge, numerous methods have been developed and studied to enhance the interpretability of such models from a human perspective.

In this thesis, we will employ some interpretability methods, namely, feature importance methods and feature effect methods. First, however, it is important to clarify what interpretability means. Interpretability is not a mathematically defined concept, nor is it a calculable metric. Instead, interpretability means that humans can easily understand the reasoning behind a model's decision. The easier a model is to understand, the more interpretable it is. Thus, some models are naturally more interpretable than others. (see Molnar, 2025, Miller, 2019, Doshi-Velez and Kim, 2017)

So, when are interpretability methods necessary? It depends on whether we are interested solely in the model's predictions or if we also want to understand the reasons behind them. If we care only about the prediction performance of the model, then we do not need interpretability methods. On the other hand, if we are interested in understanding how the model arrived at its predictions (for example, to learn more about the problem), then interpretability becomes essential. (see Molnar, 2025, Doshi-Velez and Kim, 2017)

Molnar (2025) generally distinguishes between two categories of interpretability:

Intrinsic Interpretability:

These models are inherently understandable to humans due to their design. Their internal structures are designed in such a way that they can be analyzed and understood relatively easily. Typical examples of such models are linear models or decision trees. Intrinsically interpretable models often face the problem of delivering worse prediction performance compared to complex models.

Post-hoc interpretability:

In this case, the models themselves are not inherently easily comprehensible for humans, and various methods are used to enhance their interpretability. These methods are applied after the model has been trained. The main advantage is that complex, difficult-to-understand models can be used while still obtaining insights into their decision-making process. Typical post-hoc methods, which we will also use in this thesis, are feature importance and feature effect methods.

3.2 Evaluation Metrics

First, the evaluation metrics used in the subsequent analysis of the models are presented. Evaluation metrics are necessary for assessing the overall performance of models in a clear and understandable manner, as well as for meaningfully comparing different models. This is a critical aspect of model selection.

This paper's task is to determine whether an individual is granted or denied a credit, which represents a typical standard binary classification problem. Therefore, the task at hand requires metrics that are suitable for classification.

In binary classification, there are four possible outcomes, which can be illustrated in a confusion matrix (see Table 1). In this matrix, the actual values are labeled as "True" (credit approved) or "False" (credit not approved). The model then predicts these values as "Positive" (credit approved) or "Negative" (credit not approved). The green diagonal in the table represents the two correct outcomes, while the red diagonal shows the incorrect ones. (see Tharwat, 2021)

Predicted Class \ Actual Class	True	False
	True	False
Positive	True Positive	False Positive
Negative	False Negative	True Negative

Table 1: Confusion matrix for binary classification.

True Positive (TP): An observation is a True Positive if it is actually true and is predicted to be positive.

False Positive (FP): An observation is a False Positive if it is actually false but is predicted to be positive. This outcome is also known as a Type I error.

True Negative (TN): An observation is a True Negative if it is actually false and is predicted to be negative.

False Negative (FN): An observation is a False Negative if it is actually true but is predicted to be negative. This outcome is also referred to as a Type II error.

The following evaluation metrics are derived from this matrix and are calculated using its entries.

Accuracy indicates the proportion of correctly classified observations. It is the ratio of correct predictions to the total number of observations. (see Tharwat, 2021)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

The best possible accuracy is 1, and the worst possible accuracy is 0.

Precision is defined as the number of correctly classified positive observations divided by the total number of observations predicted as positive (see Tharwat, 2021).

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

In our specific case, the term refers to the proportion of credits predicted as approved by the model that are actually worthy of approval. This is of significant importance because credits that are not correctly approved pose a considerable risk to banks and should be avoided whenever possible. The best possible value here is 1, and the worst value is 0.

Recall shows how many actual positive observations were also correctly predicted as positive by the model (see Tharwat, 2021).

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

In the context of credit approval, this measures the ratio of credits worthy of approval that were ultimately approved by the model. A high recall indicates that the bank misses out on minimal profit, as most creditworthy cases are successfully identified. The best possible recall is 1, and the worst possible recall is 0.

F₁-score is defined as the harmonic mean of precision and recall. Since it is generally difficult to achieve both high precision and high recall simultaneously, the F_1 -score serves as an effective metric to balance these two evaluation metrics. (see Tharwat, 2021)

$$F_1 - score = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}} \quad (4)$$

Specificity reveals how many actual negative observations were also correctly predicted as negative by the model (see Tharwat, 2021).

$$Specificity = \frac{TN}{TN + FP} \quad (5)$$

In our context, this metric is the ratio of correctly rejected credits to the total number of rejected credits. Again, the best possible value is 1, and the worst possible value is 0.

Finally, we turn to an evaluation metric that does not rely on the confusion matrix. The metrics previously discussed focus exclusively on the predicted class labels (credit approved or credit not approved). However, the probabilities with which a class is predicted as approved or not approved are entirely ignored. These probabilities enable us to assess a model's confidence in its prediction and are therefore also suitable for quantifying the performance of a model.

One method based on probabilities is the log loss (or cross-entropy loss), which is defined for a single observation with true label $y \in \{0, 1\}$ and predicted probability $\pi = P(y = 1)$ as follows (see Bishop, 2006):

$$L_{log}(y, \pi) = -y \log(\pi) - (1 - y) \log(1 - \pi) \quad (6)$$

The smallest possible value for the log loss is 0, while it can, in principle, become arbitrarily large. A smaller log loss indicates better performance.

3.3 Feature Importance Methods

In the later analysis, we want to achieve a more comprehensive understanding of the trained models, focusing on the non-interpretable ones. The aim is to enhance the explainability and interpretability of both the models and their respective predictions.

One approach to address this objective is to employ feature importance methods. These methods quantify the importance of features for the prediction performance of a model by assigning importance values to the individual features. The greater the importance value of a feature, the more important it is for the model.

In this paper, we use two different methods which are based on the same principle: The feature importance is measured by removing features from the data and then refitting the model.

Leave-One-Covariate-Out (LOCO)

LOCO removes only the feature of interest (FOI) X_j from the data. After that, the model is retrained with all remaining features X_{-j} , thus excluding the FOI in the new refitted model. The underlying idea behind this approach is that the LOCO score quantifies the unique contribution of a feature X_j to the prediction of the model, given the information provided by all other features. (see Lei et al., 2018)

The LOCO score can be calculated by determining the difference in expected loss of the model without the FOI $\hat{f}_{-j}^r$ and the complete original model $\hat{f}$ (see e.g., Ewald et al., 2024):

$$LOCO_j = \mathbb{E} \left[L \left(Y, \hat{f}_{-j}^r(X_{-j}) \right) \right] - \mathbb{E} \left[L \left(Y, \hat{f}(X) \right) \right], \quad (7)$$

where $\hat{f}_{-j}^r$ is based on the same learning algorithm as $\hat{f}(X)$, and Y is the target variable.

For a given dataset with n observations, we can estimate the LOCO score through Monte Carlo integration:

$$\widehat{LOCO}_j = \frac{1}{n} \sum_{i=1}^n L \left(y^{(i)}, \hat{f}_{-j}^r(x_{-j}^{(i)}) \right) - \frac{1}{n} \sum_{i=1}^n L \left(y^{(i)}, \hat{f}(\mathbf{x}^{(i)}) \right), \quad (8)$$

where $\mathbf{x}^{(i)}$ and $x_{-j}^{(i)}$ denotes the corresponding realized feature vector, and $y^{(i)}$ is the realized target value for observation $i \in \{1, \dots, n\}$.

Therefore, LOCO can be interpreted as the loss in prediction accuracy when the FOI is not considered (see Verdinelli and Wasserman, 2024).

So, how should we understand a feature with a non-zero LOCO score? Ewald et al. (2024) investigated this question and reached the following conclusion:

They found that a non-zero LOCO implies a conditional dependence between the FOI X_j and the target Y , given all remaining features X_{-j} (Result 3.3.1). Thus, a non-zero LOCO means that the FOI provides unique information about the target that the model would not have been able to access using only the remaining features.

Result 3.3.1: LOCO Interpretation: *For a cross-entropy optimal model $\hat{f}$ and a refitted optimal model $\hat{f}_{-j}^r$, it holds that*

$$LOCO_j \neq 0 \Rightarrow X_j \not\perp\!\!\!\perp Y \mid X_{-j}. \quad (9)$$

Proof. A proof for this Result can be found in Ewald et al. (2024).

Leave-One-Covariate-In (LOCI)

LOCI, in contrast, removes all features except for the FOI from the data. The model is then refitted using the FOI as the sole feature. The basic idea of LOCI is that the LOCI score now serves as a metric to quantify the contribution of a feature X_j to the prediction of the model, excluding the effect of all other features. (see Ewald et al., 2024)

The LOCI score is defined as the difference in expected loss of the optimal constant model $\hat{f}_\emptyset$ (i.e., the model without any features) and the model with the FOI as the only feature $\hat{f}_j^r$ (see e.g., Ewald et al., 2024):

$$LOCI_j = \mathbb{E} \left[L \left(Y, \hat{f}_\emptyset(X_\emptyset) \right) \right] - \mathbb{E} \left[L \left(Y, \hat{f}_j^r(X_j) \right) \right], \quad (10)$$

where $\hat{f}_j^r$ is based on the same learning algorithm as $\hat{f}(X)$, and Y is the target variable.

The LOCI score for a given dataset with n observations can be estimated using the following formula:

$$\widehat{LOCI}_j = \frac{1}{n} \sum_{i=1}^n L \left(y^{(i)}, \hat{f}_\emptyset(x_\emptyset^{(i)}) \right) - \frac{1}{n} \sum_{i=1}^n L \left(y^{(i)}, \hat{f}_j^r(x_j^{(i)}) \right), \quad (11)$$

where $x_j^{(i)}$ denotes the corresponding realized feature, and $y^{(i)}$ is the realized target value for observation $i \in \{1, \dots, n\}$.

As with LOCO, the same question arises: What does it mean if a feature has a non-zero LOCI score? Ewald et al. (2024) also studied this question and came to the following result:

In the case of LOCI, they stated that a non-zero LOCI implies an unconditional dependence between the FOI X_j and the target Y (Result 3.3.2). Therefore, a non-zero LOCI can be interpreted as an indicator that the feature contains information about the target that the model can use for prediction. However, this does not rule out the possibility that this information is also present in other features.

Result 3.3.2: LOCO Interpretation: *For cross-entropy optimal models, it holds that*

$$LOCI_j \neq 0 \Rightarrow X_j \not\perp Y. \quad (12)$$

Proof. A proof for this Result can be found in Ewald et al. (2024).

It is also important to emphasize that LOCO and LOCI do not merely estimate the feature importance for a specific model, but rather, they estimate the feature importance for the entire learning algorithm. This is because calculating LOCO and LOCI requires models to be retrained, so the resulting interpretation involves multiple models trained by the same algorithm. (see Molnar, 2025)

3.4 Feature Effect Methods

Using feature importance methods enables us to quantify the importance of individual features to a model's performance. However, in addition to assessing feature importance, we also want to understand the specific effect of a feature on a model's output and characterize the nature of the relationship (e.g., linear or non-linear) between the feature and the prediction. To address the aforementioned questions, we can employ a partial dependence plot (PDP), which was first introduced and defined by Friedman (2001).

A PDP is a method of graphically representing the marginal influence of a feature on a model's prediction. It is easy to understand, making it particularly useful for explaining complex models. A PDP illustrates how the expected prediction of a model changes when the value of a specific feature X_S is modified. The output of the model is marginalized over the distribution of the remaining features X_{-S} (i.e., the expected value is taken over the remaining features), such that the plot only depends on the values of feature X_S . Consequently, the plot only displays the relationship between X_S and the prediction. (see Friedman, 2001, Molnar, 2025)

It is also possible to visualize the PDP for two different features simultaneously (the PDP would then be a 3D-plot). In this case, X_S comprises two, rather than a single feature. Such PDPs are well-suited for illustrating the interaction between the two features in X_S . (see Molnar, 2025) The function represented by the PDP can be calculated using the following formula (see e.g., Molnar, 2025):

$$PD(X_S) = \mathbb{E}_{X_{-S}} \left[\hat{f}(X_S, X_{-S}) \right] = \int \hat{f}(X_S, X_{-S}) d\mathbb{P}(X_{-S}), \quad (13)$$

where $\hat{f}$ is the trained machine learning model.

Since we do not know the true distribution of X_{-S} , we must estimate the expected value. For a dataset with n observations, this results in (see e.g., Molnar, 2025):

$$\widehat{PD}(x_S) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_S, x_{-S}^{(i)}), \quad (14)$$

where $x_{-S}^{(i)}$ denotes the corresponding realized feature vector for observation $i \in \{1, \dots, n\}$.

In theory, this function can also be calculated for an X_S containing more than two features. However, this would result in the PDP becoming less intuitive and comprehensible visually, making PDPs of more than two features impractical. (see Molnar, 2025)

In the context of a classification task, as in our case, the PDP does not directly show the expected value of the prediction. Instead, it represents the probability of predicting a specific class. For binary classification, it is sufficient to show only a single PDP for the positive class (credit approved). (see Friedman, 2001)

3.5 Empirical Risk Minimization

Empirical Risk Minimization (ERM) is a foundational concept in machine learning. This procedure plays a central role in model selection and is decisive in determining the model received after the training process.

ERM involves finding the model from a given model class (or hypothesis space) $\mathcal{H}$ that produces the best predictions on the provided training data. This is accomplished by identifying the model with minimal risk (as measured by a loss function L) among all models in $\mathcal{H}$. Formally, this process can be represented as follows: (see Jung, 2022)

$$\mathcal{R}(f) = \mathbb{E}_{xy} [L(y, f(\mathbf{x}))] = \int L(y, f(\mathbf{x})) d\mathbb{P}(x, y), \quad (15)$$

where $\mathbf{x}$ is the feature vector, y is the target variable, and $f(\mathbf{x}) \in \mathcal{H}$.

Since the true distribution of the data is unknown, we estimate the risk using the average loss over all n individual observations:

$$\mathcal{R}_{emp}(f) = \frac{1}{n} \sum_{i=1}^n L(y^{(i)}, f(\mathbf{x}^{(i)})), \quad (16)$$

where $\mathbf{x}^{(i)}$ denotes the corresponding realized feature vector, and $y^{(i)}$ is the realized target value for observation $i \in \{1, \dots, n\}$.

The model that now provides the best predictions for the training data minimizes this expression:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \mathcal{R}_{emp}(f), \quad (17)$$

where $\hat{f}$ is now the model with minimal empirical risk.

Therefore, ERM can be used to obtain the best models for different hypothesis spaces and to compare them with each other. We can then select the most suitable model.

However, using ERM introduces a central problem: the risk of overfitting. In this special case, the model fits the training data extremely well, thereby significantly minimizing the empirical risk. Yet this can be deceptive, as the model also learns random noise and artifacts that cannot be transferred to the broader population. To detect overfitting, we can calculate the model's generalization error (GE), which indicates the model's ability to generalize. The GE can be estimated by computing the average loss of the model on new, unseen data: (see Jung, 2022)

$$\widehat{GE}(\hat{f}) = \frac{1}{m} \sum_{i=1}^m L(y^{(i)}, \hat{f}(\mathbf{x}^{(i)})), \quad (18)$$

where m is the size of the test dataset, $\mathbf{x}^{(i)}$ denotes the corresponding realized feature vector, and $y^{(i)}$ is the realized target value of the test set for observation $i \in \{1, \dots, m\}$. It is worth noting that the loss function L does not have to be the same as in Eq. (15) and Eq. (16).

At first glance, this formula is very similar to the formula for empirical risk. In principle, the two formulas are similar in that they both calculate the average loss over the data. The primary difference, however, is that the GE is calculated using additional test data that the model has not encountered during the training process.

4 Dataset

The data basis for this study is data collected under the Home Mortgage Disclosure Act (HMDA) in the United States. The HMDA contains loan-level information about mortgages from numerous financial institutions, which is subsequently published by the Consumer Financial Protection Bureau. The goal of the HMDA is to enhance transparency in lending practices within the mortgage sector, thereby providing public officials with the necessary information for decisions and other policies. Additionally, the HMDA also serves to identify and address discriminatory lending tendencies within the mortgage sector. (see *Home Mortgage Disclosure Act (HMDA)* | *FFIEC*, n.d.)

The present analysis is based on a publicly available HMDA dataset (see *HMDA Data Browser*, n.d.), which only contains data on mortgage loans from the District of Columbia for the year 2023. The dataset includes only loan applications that received explicit approval or denial. In its raw version, the dataset consists of 11531 observations (rows) and 99 variables (columns).

However, before the dataset can be used for machine learning tasks, it must first undergo some preprocessing.

First of all, it is noteworthy that the dataset contains applications with a single applicant as well as applications with two or more applicants. This factor could significantly influence the approval or rejection of the loan application. For example, applications with multiple applicants may contain more and better collateral for the credit institution, such as higher combined total income. This, in turn, can impact the approval of the loan. Consequently, a new variable was initially introduced to determine whether there is a co-applicant.

Following this, a series of columns was eliminated from the dataset. These columns were either unnecessary for the analysis or could not be used meaningfully due to a large number of missing values. The total number of variables was reduced to 16 in the process.

The remaining missing values were then examined. Where meaningful imputation (e.g., by median, mean, etc.) was possible, missing values were replaced accordingly. Observations for which imputation was not possible or appropriate were removed to ensure the integrity of the dataset. Finally, the individual variables were converted to their correct data types, and heavily skewed variables were rescaled using a log transformation. Categorical variables were transformed into factors, and numerical variables were set to a numerical data type.

The final version of the dataset, on which the analysis is being carried out, consists of a total of 11013 observations and 16 variables (15 features and the target variable).

The goal is to train different models to predict whether an applicant's credit will be approved or denied. To accomplish this, the models use a total of 15 features, such as the applicant's demographic characteristics, the purpose of the credit, and the loan amount. A more detailed explanation of the individual features and the target can be found in Appendix A. It should also be emphasized that the dataset is highly imbalanced: 74.4% of the observations are approved credits and 25.6% are rejected credits. Therefore, stratification was applied in all analyses.

The attached R code (see Appendix B) details the implementation of the described preprocessing steps.

In the following analysis, all models were trained and evaluated using the R package *mlr3* (see Lang et al., 2019). The PDPs were calculated using the R package *iml* (see Molnar et al., 2018).

5 Interpretable Models

5.1 Linear Model

In the following, we examine two interpretable models for our dataset. The first model is a logistic regression model, which is a simple and interpretable linear model used for classification tasks.

We now assume that we have a feature vector $\mathbf{x} = (1, x_1, \dots, x_p)$ with p is the number of features and a binary target variable Y , whose classes are encoded as $Y \in \{1, 0\}$. In this case, $Y = 1$ is the "positive" class, while $Y = 0$ is the "negative" class. The fundamental principle of logistic regression is to predict the probability $P(Y = 1|\mathbf{x}) = P(\mathbf{x})$ and thereby ultimately return a specific class as prediction. Just as in standard linear regression, one could assume a linear relationship between the probabilities and the features $P(\mathbf{x}) = \beta^\top \mathbf{x}$. The inherent problem with this approach is that probabilities greater than one or less than zero would then be possible. However, this is obviously not meaningful, as probabilities naturally only range from 0 to 1. To address this problem, logistic regression uses the logistic function h to map the probabilities to the interval $[0, 1]$. (see Fahrmeir et al., 2013)

$$P(\mathbf{x}) = h(\beta^\top \mathbf{x}) = \frac{\exp(\beta^\top \mathbf{x})}{1 + \exp(\beta^\top \mathbf{x})} = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}{1 + \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}, \quad (19)$$

where $\beta = (\beta_0, \beta_1, \dots, \beta_p)$ are the parameters of the model.

In the end, the model (i.e., the set of parameters) that minimizes the empirical risk is selected:

$$\arg \min_{\beta} \mathcal{R}_{emp}(\beta) = \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n L(y^{(i)}, P(\mathbf{x}^{(i)}|\beta)), \quad (20)$$

where n is the number of observations and L is a loss function.

Nevertheless, why is logistic regression now considered an interpretable and linear model, even though its basic structure (Eq. 19) is neither linear nor necessarily easy to understand?

This is because the model is linear when viewed on a different scale. Logistic regression models a linear relationship between the features and the log odds (Eq. 21). Consequently, features exert an exponential influence on the standard odds (Eq. 22). If the value of a feature changes, the subsequent effect on the prediction becomes readily interpretable in terms of the odds. For instance, the log odds increase by β_1 when feature x_1 is incremented by one unit. At the same time, if feature x_1 increases by one unit, the odds change by a factor of $\exp(\beta_1)$. Hence, changes in the values of the features have a multiplicative effect on the odds. (see Fahrmeir et al., 2013)

$$\log \left(\frac{P(\mathbf{x})}{1 - P(\mathbf{x})} \right) = \beta^\top \mathbf{x} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p \quad (21)$$

$$\left(\frac{P(\mathbf{x})}{1 - P(\mathbf{x})} \right) = \exp(\beta^\top \mathbf{x}) = \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p), \quad (22)$$

where $\frac{P(\mathbf{x})}{1 - P(\mathbf{x})}$ is called the odds.

To better illustrate this, let us look at a brief hypothetical example with specific numbers for the parameters.

$$P(Y = 1|\mathbf{x}) = 0.5 + 2.7x_1$$

Here, the parameters β_0 and β_1 are set to 0.5 and 2.7, respectively. If x_1 increases by one unit, the log odds increase by 2.7. Accordingly, the odds change by a factor of $\exp(2.7)$.

Overall, this example demonstrates that logistic regression is a highly interpretable model.

Finally, we examine the logistic regression model for our HMDA dataset. In this context, the "positive" class refers to loans that were approved, while the "negative" class denotes loans that were rejected.

First, we consider the overall performance of the model (see Table 2). The evaluation metrics were calculated using subsampling with a total of 50 repetitions and a training data share of 75%.

Accuracy	Precision	Recall	Specificity	F1-score	log loss
0.838	0.854	0.944	0.532	0.897	0.37

Table 2: Evaluation results for the logistic regression model.

The model demonstrates satisfactory overall performance, achieving an accuracy of approximately 84%. This means that about 84% of the observations were correctly predicted by the logistic regression model. However, due to the highly imbalanced distribution of the target variable, the informativeness of this value in isolation is limited. Examining the precision reveals that around 85% of the loans predicted as approved by the model were indeed correctly classified. Conversely, this also means that 15% of the loans approved by the model were incorrectly classified, representing a potential risk for the credit institution.

Furthermore, a recall rate of around 94% demonstrates the model's effectiveness in correctly classifying loans that were actually approved. The situation is quite different for rejected loans. Here, the observed specificity is approximately 53%. This means that around half of all the actual rejected loans were incorrectly classified, which also corresponds to the performance of simple random guessing.

In general, the logistic model has clear limitations when a loan has actually been rejected. It performs considerably better when it comes to predicting actual approved loans.

Next, we analyze the feature importance values of the logistic regression model (see Figure 1). These values were determined on a test set comprising 25% of the data, using log loss as metric, averaged over 50 repetitions.

It is immediately apparent that both LOCO and LOCI identify "debt_income_ratio" as the most important feature and "loan_purpose" as the second most important. Notably, "loan_purpose" appears to be relatively more important according to LOCI than according to LOCO. This suggests that these two features have both a conditional and an unconditional association with the target variable.

After that, some differences become apparent. According to LOCI, the next most important features are "loan_amount," "lien_status," "income," and "race." However, according to LOCO, all these features are considered completely irrelevant. In fact, "income" and "loan_amount" are two of the least important features according to LOCO. This indicates that these features are unconditionally related to the target but not conditionally. Consequently, the information contained within these features is useful for prediction, yet it is also captured by other features. According to LOCO, the feature "property_value" is also important, indicating a conditional relation to the target.

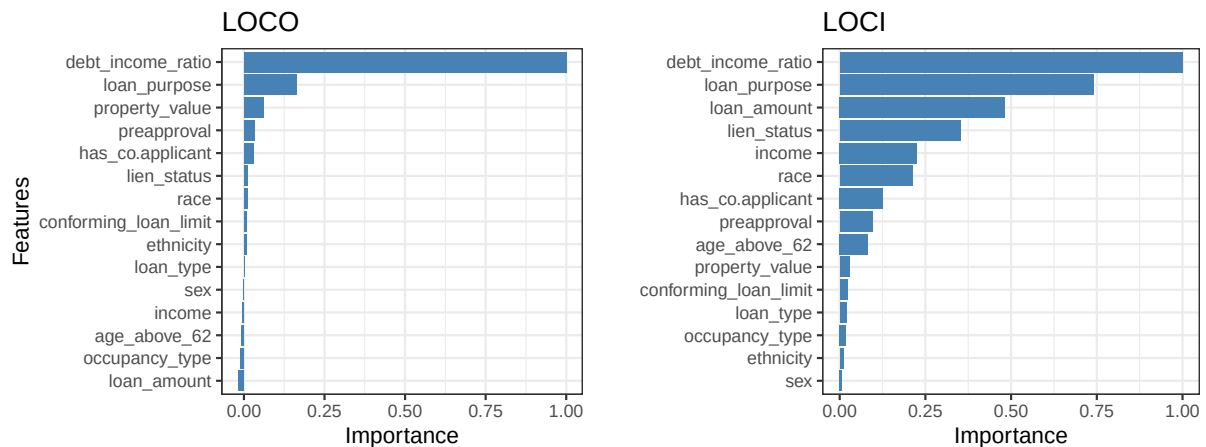


Figure 1: Feature importance values for the logistic regression model; values are relative to the most important feature.

Both methods consider "preapproval" and "has_co.applicant" to be important, although LOCO assigns a higher rank to them than LOCI. Therefore, these two features appear to have both a conditional and an unconditional relationship with the target variable. All other features are relatively unimportant according to both methods.

We now proceed to examine PDPs for a total of four features (see Figure 2). To calculate the PDPs, an additional model was trained on 75% of the data, while the remaining 25% of the data was used to determine the PDPs. The PDPs show the predicted probability of loan approval.

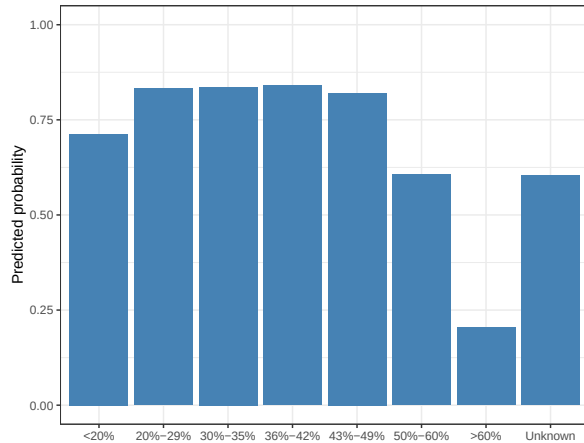
Figure 2a displays the effect of the various categories of "debt_income_ratio." As can be seen, the probability first increases and then remains roughly constant at approximately 82% across the entire range from 20% to 49%. Once the ratio reaches 50%–60%, the probability drops significantly to around 60%. If the ratio exceeds 60%, the probability drops further to around 20%. In cases where the ratio is unknown, the probability is centered at approximately 60%. Overall, an excessively high debt-income ratio has a strongly negative effect on the probability of loan approval.

Figure 2b shows the effect of the feature "loan_purpose." The category "Home purchase" is particularly noteworthy, as it is the category with the highest probability of loan approval (~83%). Interestingly, all other categories exhibit a similar probability, at around 63%, except for the "Cash-out" category, which is around 70%. Once again, we observe a strong influence of the different categories on the approval probability.

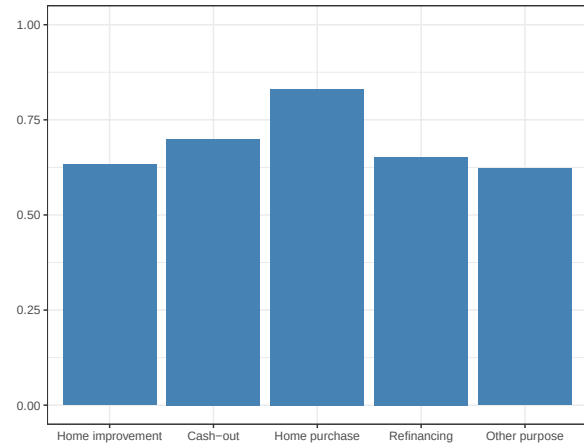
Figure 2c illustrates the effect of "race." In contrast to the first two features, the effect here is generally much smaller. This observation is consistent with the ranking of the importance values according to both LOCO and LOCI. Applicants who are Black or African American generally have a slightly lower probability of loan approval compared to the other groups. For this group, the probability is estimated to be approximately 70%, while for all other races, the probability is higher, at around 75%. White applicants have the highest probability of loan approval. Thus, a slight disadvantage associated with the applicant's race can be observed.

Figure 2d presents the effect of "preapproval." This feature is particularly noteworthy. When a preapproval has been requested, the probability of loan approval is 100%. In instances where a preapproval has not been requested, the probability drops to around 75%. This finding is

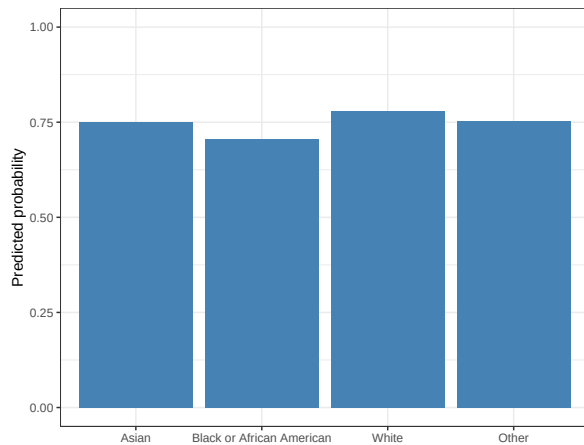
interesting because both LOCO and LOCI considered this feature to be relatively unimportant despite the fact that it appears to provide reliable predictive information.



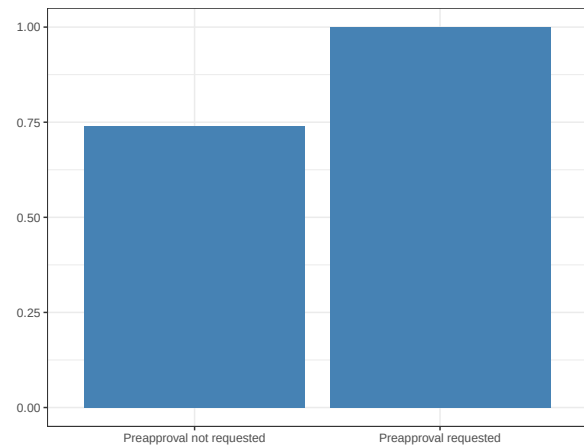
(a) debt_income_ratio



(b) loan_purpose



(c) race



(d) preapproval

Figure 2: Feature effects for the logistic regression model.

5.2 Classification Tree

The next model we discuss for our dataset is a classification and regression tree (CART) (see Breiman et al., 1984). In our analysis, we exclusively use classification trees. As with the logistic regression model discussed previously, this model is also considered interpretable.

We now assume a feature vector $\mathbf{x} = (x_1, \dots, x_p)$ and a binary target variable y . A classification tree is suitable for directly predicting class labels; however, it is also capable of returning probabilities for each class. The basic idea behind a classification tree is to use a series of decision rules to iteratively partition the data into smaller, more homogeneous groups, which can ultimately be assigned predominantly to a single class.

Now, we examine the process of constructing such a tree. The tree begins at the root node, which initially contains all observations. This node is then divided into two smaller child nodes using binary splitting. The splitting rule used to separate the data (Eq. 23) is chosen such that the resulting split minimizes the risk (Eq. 25). This process is then repeated recursively for each of the resulting nodes. This procedure is also referred to as recursive binary splitting, and its implementation ensures that the resulting classification tree will take the form of a binary tree. This iterative process persists until a predetermined stopping criterion is met (e.g., maximum depth, insufficient data in a node, etc.). The final nodes, called leaves, are those that are not split further. Finally, the leaves determine the exact prediction values. As hard labels, each leaf predicts the class that occurs most frequently within it, while the probabilities are given by the relative frequencies of the different classes in each leaf. The entire process can also be formalized in terms of ERM as follows: (James et al., 2021, Hastie et al., 2009)

A parent node $\mathcal{N}$ is split with respect to a feature x_j at a split point s , resulting into two child nodes:

$$\mathcal{N}_1 = \{(\mathbf{x}, y) \in \mathcal{N} : x_j < s\} \text{ and } \mathcal{N}_2 = \{(\mathbf{x}, y) \in \mathcal{N} : x_j \geq s\}. \quad (23)$$

The risk of the split is then given by:

$$\mathcal{R}(\mathcal{N}, j, s) = \mathcal{R}_1(\mathcal{N}_1) + \mathcal{R}(\mathcal{N}_2) = \sum_{(\mathbf{x}, y) \in \mathcal{N}_1} L(y, P_{\mathcal{N}_1}) + \sum_{(\mathbf{x}, y) \in \mathcal{N}_2} L(y, P_{\mathcal{N}_2}), \quad (24)$$

where $P_{\mathcal{N}_1}$ and $P_{\mathcal{N}_2}$ are predicted class probabilities in the respective child node and L represents a loss function.

The optimal way to split a node minimizes the following expression:

$$\arg \min_{j, s} \mathcal{R}(\mathcal{N}, j, s). \quad (25)$$

As previously stated, a classification tree is an interpretable model. Each path in the tree represents a sequence of decision rules that collectively yield a specific prediction. Every decision rule is clearly defined, making each path—and thus the entire tree—easily understandable. Another advantage of a classification tree is its ability to be represented visually as a tree diagram, which makes interpreting smaller trees even easier.

To better illustrate this, let us look at a brief graphical example of a heavily pruned classification tree for our HMDA dataset (see Figure 3).

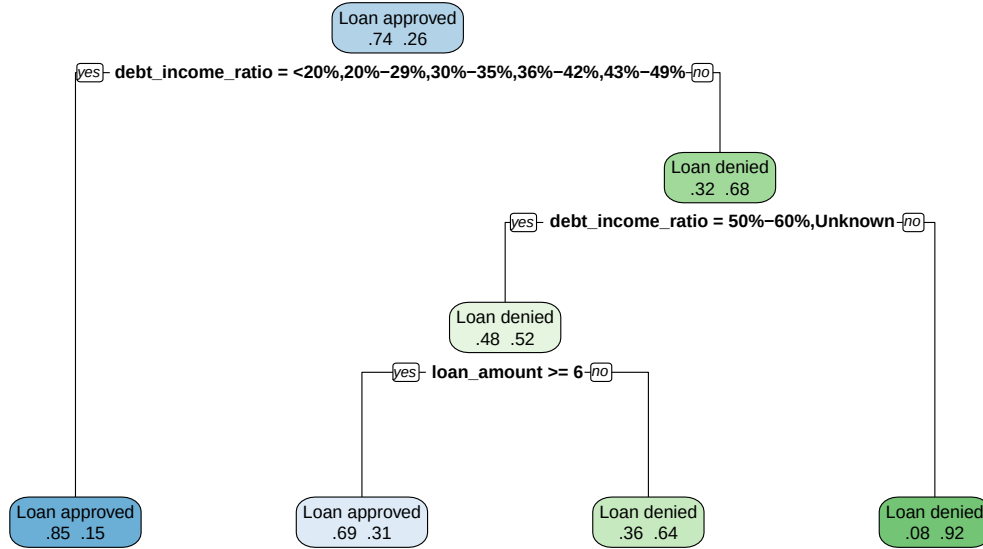


Figure 3: Classification tree predicting loan approval based on "debt_income_ratio" and "loan_amount." Probabilities at terminal nodes indicate class distributions.

In this example, the tree contains a total of four unique paths. At each leaf node, both the predicted class label and the predicted probabilities are displayed. For instance, if an observation has "debt_income_ratio = 50%–60%" and "loan_amount = 7," it would arrive at the second leaf and the model would predict "Loan approved" with a probability of 69%.

Finally, we examine the (complete) classification tree for our HMDA dataset. First, hyperparameter tuning was performed for the parameters "cp," "maxdepth," "min.bucket," and "minsplit." To this end, random search with 500 evaluations was used. Each evaluation was trained on 66.7% of the data using a holdout split and evaluated on the remaining data. The optimal hyperparameter configuration was found to be "cp = 0.0014," "maxdepth = 24," "min.bucket = 15," and "minsplit = 63."

First, we examine the overall performance of the classification tree (see Table 3). The evaluation metrics were calculated using subsampling with a total of 50 repetitions and a training data share of 75%. The model was trained using the optimal hyperparameters.

Accuracy	Precision	Recall	Specificity	F1-score	log loss
0.842	0.858	0.944	0.545	0.899	0.384

Table 3: Evaluation results for the classification tree.

The model also demonstrates satisfactory performance, with an accuracy of approximately 84%. This indicates that 84% of the data was correctly classified. Looking at the precision, we see that about 86% of the loans predicted as approved were indeed correctly classified. Conversely, this implies that around 14% were misclassified as approved, which represents a potential risk for the credit institution.

Turning our attention to recall and specificity reveals that, with a recall of approximately 94%, virtually all actual approved loans were correctly classified. In contrast, specificity yields a lower value of approximately 54%. This suggests that about half of the actual rejected loans were misclassified, which corresponds to simple random guessing and is, of course, undesirable.

Overall, the classification tree clearly has weaknesses when a loan has been actually rejected. However, its performance is much better for actual approved loans.

Next, we analyze the feature importance values of the classification tree (see Figure 4). These values were determined on a test set comprising 25% of the data, using log loss as metric and averaging over 50 repetitions. For training the model, the optimal hyperparameters were used.

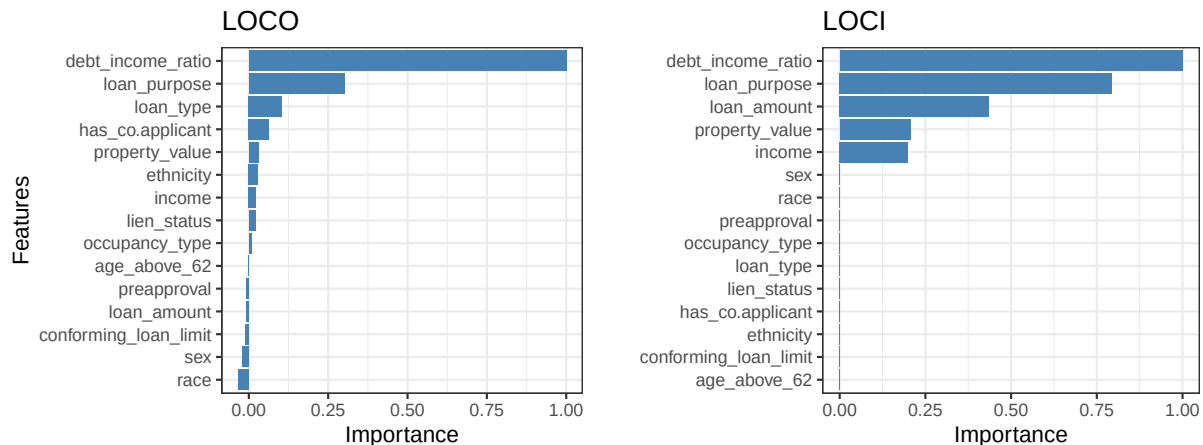


Figure 4: Feature importance values for the classification tree; values are relative to the most important feature.

Looking at LOCI, we immediately notice something interesting: Most features have an importance value of zero. Only five features have a non-zero importance value. To explain this phenomenon, it is necessary to review the underlying mechanism of LOCI. LOCI always processes each feature individually to train a model and derive the LOCI score. So, for features with a zero LOCI score, the model was unable to find a meaningful split, resulting in a trivial tree and, consequently, a zero LOCI score.

Again, both LOCO and LOCI identify "debt_income_ratio" as the most important feature and "loan_purpose" as the second most important, with "loan_purpose" being comparatively more important according to LOCI than according to LOCO. Based on this model, we can also conclude that these two features have a conditional as well as an unconditional relation to the target.

LOCI also identifies "loan_amount," "property_value," and "income" as important features. Meanwhile, according to LOCO, these three features are somewhat irrelevant. Therefore, these features have an unconditional association with the target, though not a conditional one. According to LOCO, the subsequent important features are "loan_type" and "has_co.applicant," indicating a conditional relation with the target. As previously discussed, all these features have

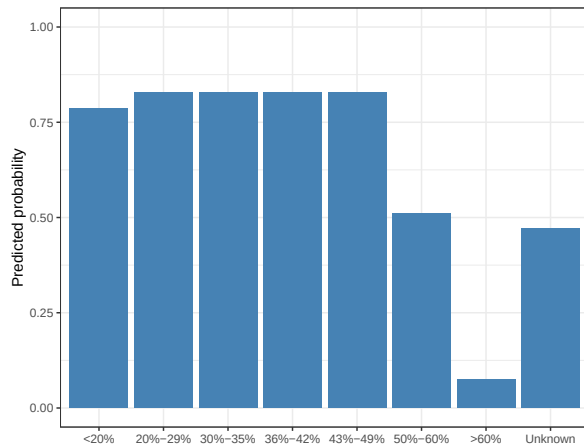
a zero LOCI score. All other remaining features are also considered relatively unimportant according to LOCO.

Finally, we proceed to examine PDPs for a total of five features (see Figure 5). To calculate the PDPs, a new model with the optimal set of hyperparameters was trained on 75% of the data, while the remaining 25% of the data was used to determine the PDPs. The PDPs show the predicted probability of loan approval. In addition, the split points of the classification tree are visible as jump points in the graphs.

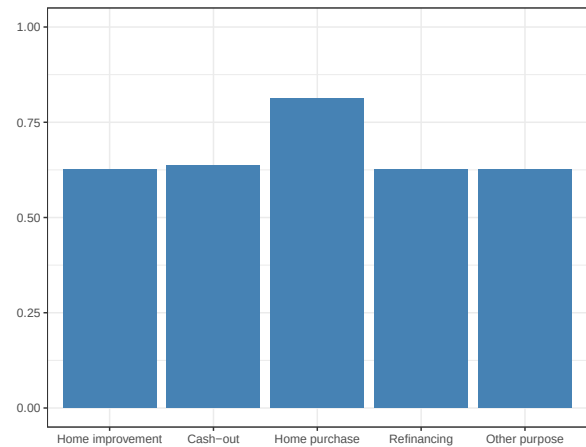
Figure 5a illustrates the impact of the various "debt_income_ratio" categories. As shown, the probability of loan approval rises slightly at first and remains constant in the interval from 20% to 49% at a value of 82%. When entering the 50%–60% range, the probability drops significantly to about 50%. For values above 60%, the probability drops even further to approximately 7%. In unknown cases, the probability averages around 47%. Thus, consistent with the feature importance results, the "debt_income_ratio" has a strong influence in this model.

Figure 5b shows the effect of the different "loan_purpose" categories. The "Home purchase" category has the highest approval rate, at about 80%. All other categories have very similar approval rates, around 62%, with the "Cash-out" category being slightly higher. Once again, we observe a strong influence of this feature, consistent with the importance values.

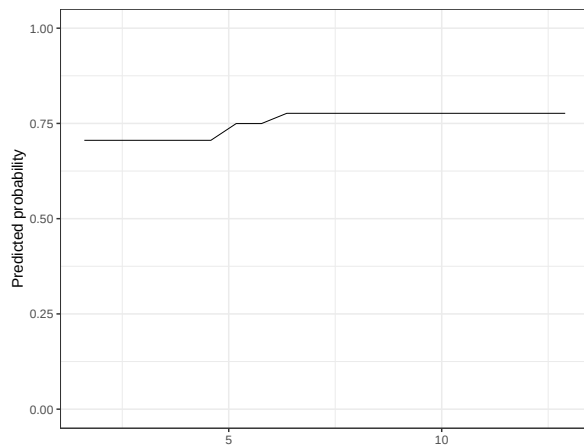
Figures 5c, 5d, and 5e illustrate the features "loan_amount," "property_value," and "income," respectively. The three plots show a similar trend: For all three features, the probability increases sharply in the interval from 3 to 7. For "loan_amount" and "property_value," the probability then levels off at approximately 78%. In contrast, the effect of "income" differs, as the probability first declines again before stabilizing at approximately 73%. Despite the relatively low LOCO scores of these features, a substantial effect on the prediction can be observed.



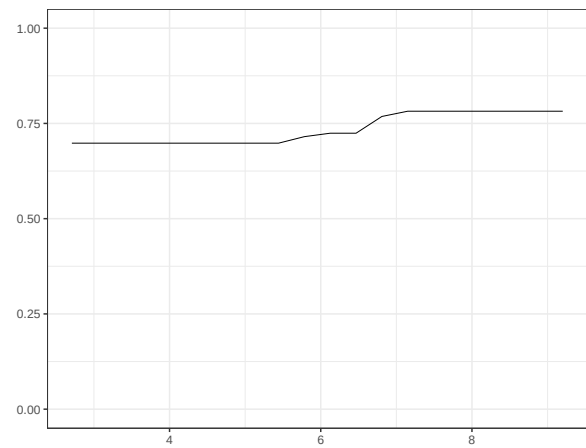
(a) debt_income_ratio



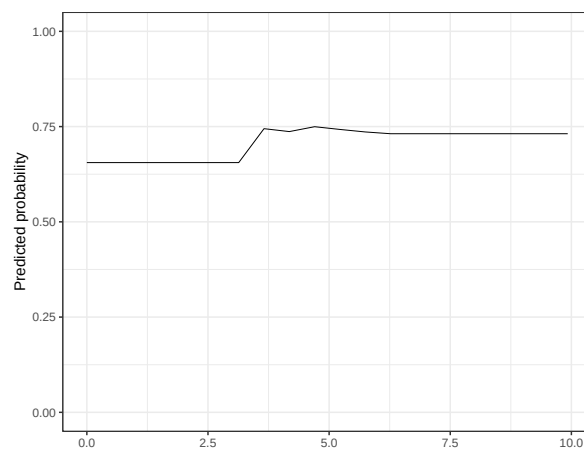
(b) loan_purpose



(c) loan_amount



(d) property_value



(e) income

Figure 5: Feature effects for the classification tree.

6 Non-Interpretable Models

6.1 Random Forest

In the following two sections, we examine two non-interpretable models for our dataset. The first non-interpretable model is the widely used random forest (see Breiman, 2001a), which is essentially an extension of CARTs. In this section, the focus will be on random forests for classification tasks.

One major drawback of CARTs (see Section 5.2) is their sensitivity to variations in the data, which reflects their limited robustness. Even minor changes in the training data can result in substantial differences in the estimated trees. Thus, the structure of the tree can vary greatly, even if the underlying training data is only slightly different. This leads to high variance in predictions and comparatively poor performance, which is undesirable. (see Hastie et al., 2009)

This issue is addressed by random forests, which are based on bagging, an ensemble method. Ensemble methods combine multiple (sometimes different) simple models (called base learners) into one large, powerful model that usually performs better than a single base learner. Bagging is a homogeneous ensemble method (i.e., it uses only one base learner class) in which M base learners are trained on different subsets of the training data. The final prediction is then the class most commonly predicted by the M base learners. Each of the M training sets for the base learners is of identical size and is drawn from the original training data using bootstrapping with replacement. This can result in observations occurring multiple times in a training set while others do not occur at all. (see James et al., 2021)

A random forest is based on bagging and uses CARTs (in this case, classification trees) as base learners. The M different trees are constructed following the procedure described in Section 5.2. However, there are two differences: First, as previously stated, the training sets for the individual trees are derived using bootstrapping, meaning that each tree is trained on a different training set. Second, at each split, each tree can only utilize a randomly selected subset of the available features for finding the best split. This increases the diversity among the trees and reduces their correlation. The prediction of the random forest is the class most commonly predicted by the M trees. The probability of each class is determined by its relative frequency among the predictions of the M trees. (see Breiman, 2001a)

In contrast to the classification tree, however, the random forest is no longer interpretable. The internal decision-making process of the model is no longer readily comprehensible, and thus, the rationale for a particular prediction is not directly apparent. This is because the prediction is now the result of many different trees. To comprehend the impact of a specific feature, it is necessary to traverse all M trees and evaluate their individual effects. For large values of M , this process can get quite time-consuming.

Now, we apply the random forest to our HMDA dataset. Prior to this, hyperparameter tuning was performed for the parameters "num.trees," "replace," "sample.fraction," "mtry.ratio," and "min.node.size." To this end, random search with 500 evaluations was used. Each evaluation was trained on 66.7% of the data using a holdout split and evaluated on the remaining data. The optimal hyperparameter configuration was found to be "num.trees = 877," "replace = TRUE," "sample.fraction = 0.949," "mtry.ratio = 0.375," and "min.node.size = 20."

First, we examine the overall performance of the random forest with optimal hyperparameters (see Table 4). The evaluation metrics were calculated using subsampling with a total of 50 repetitions and a training data share of 75%.

Accuracy	Precision	Recall	Specificity	F1-score	log loss
0.848	0.862	0.948	0.559	0.903	0.359

Table 4: Evaluation results for the random forest.

The model achieves an accuracy of approximately 85%, meaning that 85% of the observations are correctly classified. Overall, this model performs relatively well. Examining the precision shows that around 86% of the predicted approved loans were also correctly classified. This also means that 14% were misclassified, which may pose a potential risk for the credit institution. Regarding the recall, approximately 95% of loans that were actually approved were also correctly classified. The situation is quite different, however, when it comes to specificity. The model's specificity is approximately 56%, meaning that 44% of the actual rejected loans were incorrectly approved by the model. This is suboptimal, considering that these cases should be reduced as much as possible.

In summary, the evaluation of the random forest algorithm reveals its deficiencies in accurately predicting loans that were actually rejected. However, the model has no difficulties with loans that were actually approved.

Next, we analyze the feature importance values of the classification tree (see Figure 6). These values were determined on a test set comprising 25% of the data, using log loss as metric and averaging over 50 repetitions. For training the model, the optimal hyperparameters were used.

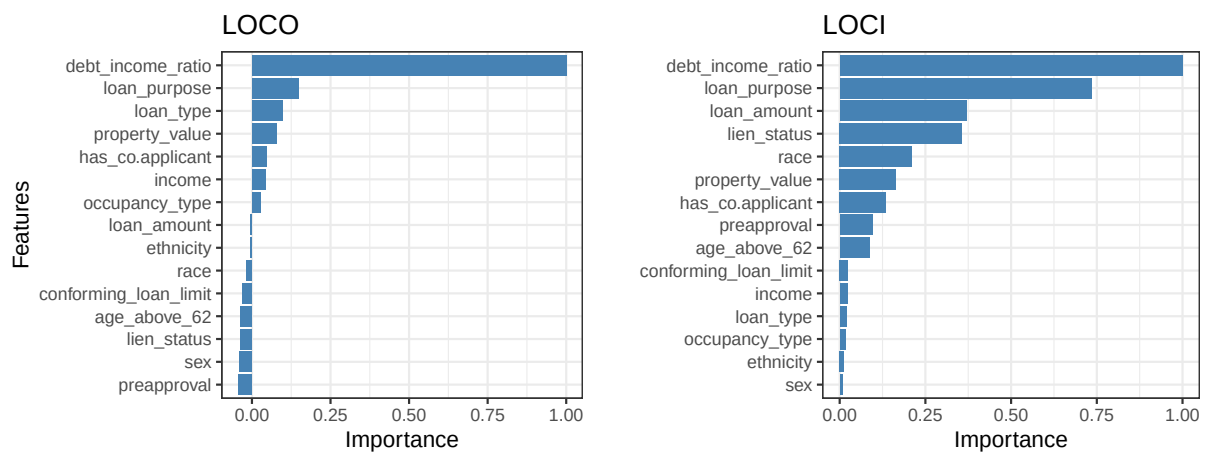


Figure 6: Feature importance values for the random forest; values are relative to the most important feature.

Similar to the previously analyzed models, it is immediately apparent that both LOCO and LOCI identify "debt_income_ratio" as the most important feature, followed by "loan_purpose" as the second most important feature. However, "loan_purpose" is relatively more important according to LOCI than according to LOCO. This finding indicates that these two features have both a conditional and an unconditional relationship to the target.

From this point onward, the two methods differ. According to LOCO, "loan_type" is now the next most important feature, while according to LOCI, the next most important feature is "loan_amount." Interestingly, each of these features is considered unimportant by the other method. From this, we can conclude that "loan_type" has a conditional relationship to the target and "loan_amount" has an unconditional relationship to the target.

According to LOCO, the features "property_value," "has_co.applicant," "income," and "occupancy_type" are also important. LOCI also identifies "property_value" and "has_co.applicant" as relevant, while it considers "income" and "occupancy_type" as unimportant. This means that "property_value" and "has_co.applicant" have both a conditional and an unconditional relationship to the target, while "income" and "occupancy_type" only exhibit a conditional relationship. LOCI also identifies "lien_status," "race," "preapproval," and "age_above_62" as important features. In contrast, LOCO considers these features unimportant, and they even have negative LOCO scores (meaning they worsen the model according to LOCO). This indicates that these features have an unconditional association with the target.

Finally, we proceed to examine PDPs for a total of four features (see Figure 7). To calculate the PDPs, an additional new model with the optimal set of hyperparameters was trained on 75% of the data, while the remaining 25% of the data was used to determine the PDPs. The PDPs show the predicted probability of loan approval.

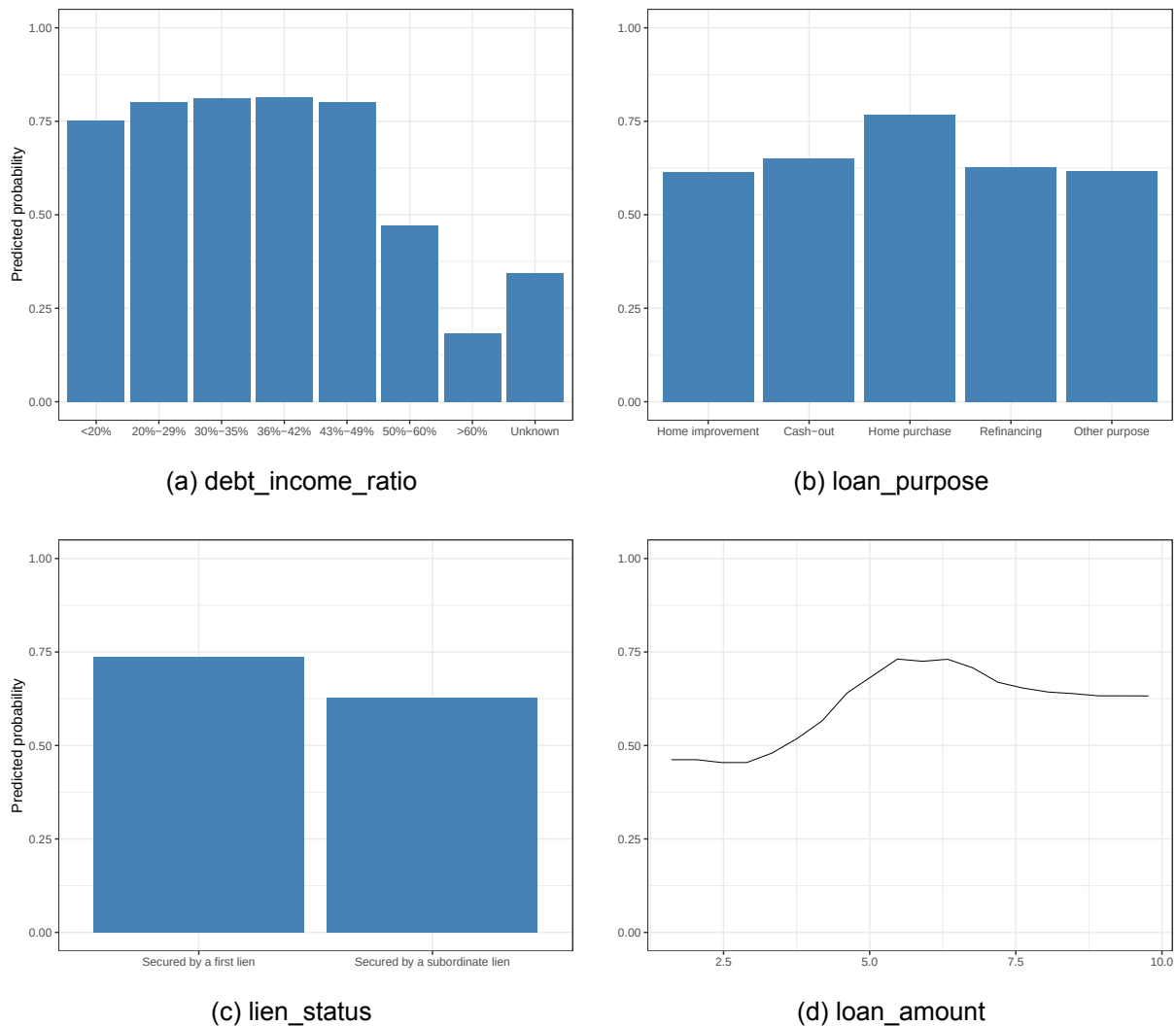


Figure 7: Feature effects for the random forest.

In Figure 7a, we see the effect of each category of "debt_income_ratio." We observe that the probability rises slightly at first and then levels off at approximately 80% across several categories.

Once the 50%–60% range is reached, the probability drops significantly to around 47%. If the ratio exceeds 60%, the probability drops further to about 18%. In the unknown cases, the probability is approximately 34%. Consistent with the results of the feature importance methods, "debt_income_ratio" is also observed to have a notable impact on the model.

Figure 7b shows the effect of the different categories of "loan_purpose." The "Home purchase" category has the highest approval rate, at about 77%. All other categories are very similar, with a value of approximately 62%, except for the category "Cash-out," which has a slightly higher value. Here, as well, the feature has a strong influence on the model, which is also reflected in the feature importance values.

In Figure 7c, we see the effect of the categories of "lien_status." The probability of a loan being approved is significantly higher if the loan is secured by a first lien. In this case, the probability is about 74%. If the loan is secured by a subordinate lien, the probability is significantly lower, at about 63%.

In Figure 7d, we see the effect of "loan_amount." Initially, the probability of loan approval is centered at approximately 46%. However, this probability then rises quickly, reaching a peak at approximately 5.5 with a probability of around 73%. After that, the probability drops again and eventually settles back down to approximately 63%.

6.2 Support Vector Machine

The next non-interpretable model we analyze is the support vector machine (SVM) (see Boser et al., 1992). In this section, we examine both a linear SVM and a non-linear SVM. As in the previous sections, the focus remains exclusively on SVMs for classification tasks.

We now assume a feature vector $\mathbf{x} = (x_1, \dots, x_p)$ and a binary target variable y , whose classes are encoded as $y \in \{-1, 1\}$. The basic idea behind an SVM is to find an optimal hyperplane (decision boundary) that separates the two classes of y with the maximum possible distance to the observations. Predictions are then made based on the position of the data points relative to this hyperplane.

The simplest version of an SVM is the linear hard margin SVM. In this case, it is assumed that the data is linearly separable, meaning that a hyperplane exists which can perfectly separate the two classes without any error. The margin is defined as the minimum (perpendicular) distance between an observation and the hyperplane (Eq. 26), i.e., the distance from the hyperplane to the nearest training point. The objective of a hard margin SVM is to determine a perfectly separating hyperplane while simultaneously maximizing the margin (Eq. 27). In other words, we seek a hyperplane that separates the data and, at the same time, maximizes the distance to the nearest observations. The procedure is formally described as follows: (Boser et al., 1992)

The margin M is defined as:

$$M = \min_i d(f, \mathbf{x}^{(i)}) = y^{(i)} \frac{f(\mathbf{x}^{(i)})}{\|\beta\|} = y^{(i)} \frac{\beta^\top \mathbf{x}^{(i)} + \beta_0}{\|\beta\|}, \quad (26)$$

where $f = \beta^\top \mathbf{x} + \beta_0$ is the hyperplane.

In dual form, the linear hard margin SVM optimizes the following expression:

$$\begin{aligned} \max_{\alpha \in \mathbb{R}^n} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y^{(i)} = 0, \\ & 0 \leq \alpha_i \quad \forall i \in \{1, \dots, n\}, \end{aligned} \quad (27)$$

where n is the number of observations and α_i are the Lagrange multipliers.

The aforementioned procedure is based on the assumption that the data can be perfectly separated by a hyperplane. However, in most practical applications, this assumption does not hold. If the data is not linearly separable, then there is no perfectly separating hyperplane; therefore, the hard margin SVM cannot find a solution. To address this issue, the procedure has been extended to the linear soft margin SVM (Eq. 28). The objective remains unchanged: We want to find a hyperplane that separates the data while maximizing the margin. The difference is that some violations of the margin are now allowed. This means that observations can now have a distance to the hyperplane that is less than the margin, and some points may even be located on the wrong side of the hyperplane, i.e., they may be misclassified. This approach ensures that a solution can be found even when the data is not linearly separable. In dual form, the linear soft margin SVM optimizes the following expression: (see Cortes and Vapnik, 1995)

$$\begin{aligned}
\max_{\alpha \in \mathbb{R}^n} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\
s.t. \quad & \sum_{i=1}^n \alpha_i y^{(i)} = 0, \\
& 0 \leq \alpha_i \leq C \quad \forall i \in \{1, \dots, n\},
\end{aligned} \tag{28}$$

where C is the cost parameter of the model which controls the frequency and size of margin violations.

Even with the implementation of the soft margin SVM, our modeling capabilities remain limited. We still assume that the data can be reasonably well, though not perfectly, separated by a linear function (i.e., a hyperplane). In applications where this assumption does not hold, a linear soft margin SVM may not yield a satisfactory solution. To address this challenge, the approach has been further extended to the (general) SVM. The major change is that the input space is transformed into a higher-dimensional feature space through the use of a kernel. Then, in this new feature space, a hyperplane is constructed to separate the data using the soft margin SVM formulation (Eq. 29). In this higher-dimensional feature space, the decision boundary is linear; however, in the original input space, the boundary is no longer linear. This so-called kernel trick allows us to construct non-linear SVMs, thus allowing the SVM to learn more flexible decision boundaries. In dual form, the (general) SVM optimizes the following expression: (see Boser et al., 1992, James et al., 2021)

$$\begin{aligned}
\max_{\alpha \in \mathbb{R}^n} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \\
s.t. \quad & \sum_{i=1}^n \alpha_i y^{(i)} = 0, \\
& 0 \leq \alpha_i \leq C \quad \forall i \in \{1, \dots, n\},
\end{aligned} \tag{29}$$

where k is the kernel function.

In the case of a linear SVM, the kernel k is simply the standard scalar product, i.e.:

$$k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle. \tag{30}$$

In the case of a nonlinear SVM, the kernel k can take many different forms. The kernel we use is the sigmoid kernel, i.e.:

$$k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \tanh(\gamma \mathbf{x}^{(i)\top} \mathbf{x}^{(j)} + b), \tag{31}$$

where γ and b (later called `coef0`) are parameters of the model.

Looking at the mathematical formulation of an SVM reveals that it is a non-interpretable model. With the introduction of soft margin SVMs and kernels, it has become very challenging to understand how the model constructs its decision boundary or how changes in the features influence this boundary.

Finally, we examine the linear SVM for our HMDA dataset. First, hyperparameter tuning was performed for the parameter "cost." To this end, random search with 500 evaluations was used. Each evaluation was trained on 66.7% of the data using a holdout split and evaluated on the remaining data. The optimal hyperparameter was found to be "cost = $6.74 * 10^{-5}$."

First, we look at the overall performance of the linear SVM with the optimal hyperparameter (see Table 5). The evaluation metrics were calculated using subsampling with a total of 50 repetitions and a training data share of 75%.

Accuracy	Precision	Recall	Specificity	F1-score	log loss
0.833	0.848	0.945	0.507	0.894	0.396

Table 5: Evaluation results for the linear SVM.

With an accuracy of approximately 83%, the model correctly classifies a substantial proportion of observations. Like the other models, it performs relatively well. However, it should be noted that approximately 15% of the loans predicted as approved were incorrectly classified and should have been rejected, as indicated by a precision of about 85%.

According to the recall, approximately 95% of the loans that were actually approved were also predicted to be approved by the model. Thus, virtually all approved loans were correctly classified. However, the model only achieved a specificity of about 51%, meaning that approximately half of the actual rejected loans were misclassified.

Overall, the linear SVM also performs poorly when it comes to loans that should be rejected. However, the model has no difficulty with actual approved loans.

Next, we analyze the feature importance values of the linear SVM (see Figure 8). These values were determined on a test set comprising 25% of the data, using log loss as metric and averaging over 50 repetitions. The model was trained using the optimal hyperparameters.

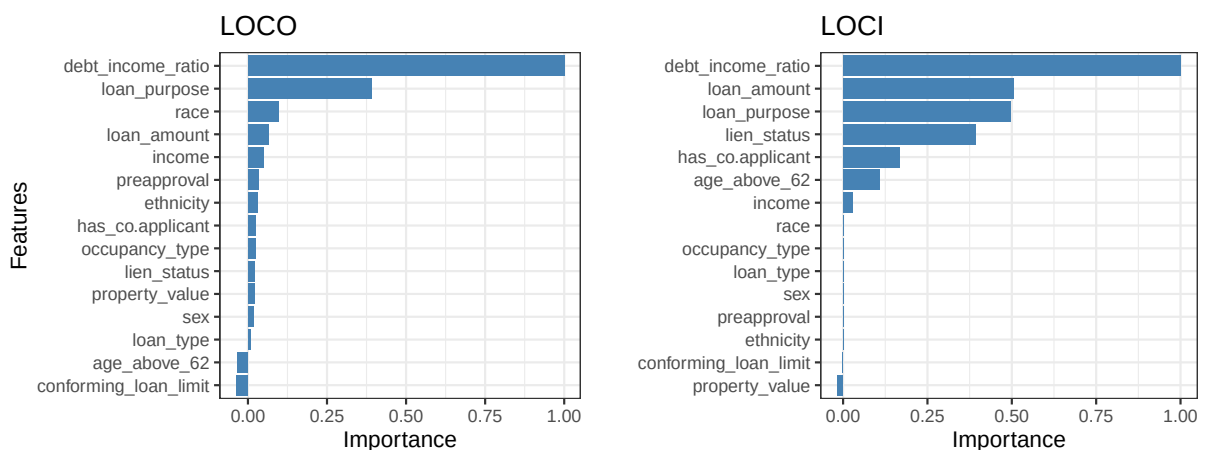


Figure 8: Feature importance values for the linear SVM; values are relative to the most important feature.

Both methods consider "debt_income_ratio" to be the most important feature. Therefore, this feature is both conditionally and unconditionally related to the target.

At this point, the methods diverge. According to LOCO, the next most important feature is "loan_purpose," whereas according to LOCI, the next most important is "loan_amount." However, both features are also considered important by the other method, and LOCI even assigns nearly equivalent importance to "loan_amount" and "loan_purpose." Thus, these two features have both conditional and unconditional relationships to the target.

Furthermore, the feature "race" is also important according to LOCO, yet it is considered irrelevant by LOCI. Consequently, it exhibits a conditional relationship to the target.

In contrast, LOCI considers the features "lien_status," "has_co.applicant," and "age_above_62" to be important for the model, while LOCO does not. Consequently, these features exclusively have an unconditional relationship to the target. All remaining features are considered unimportant by both LOCO and LOCI.

Now, we proceed to examine PDPs for a total of two features (see Figure 9). To calculate the PDPs, an additional new model with the optimal hyperparameter was trained on 75% of the data, while the remaining 25% of the data was used to determine the PDPs. The PDPs show the predicted probability of loan approval.

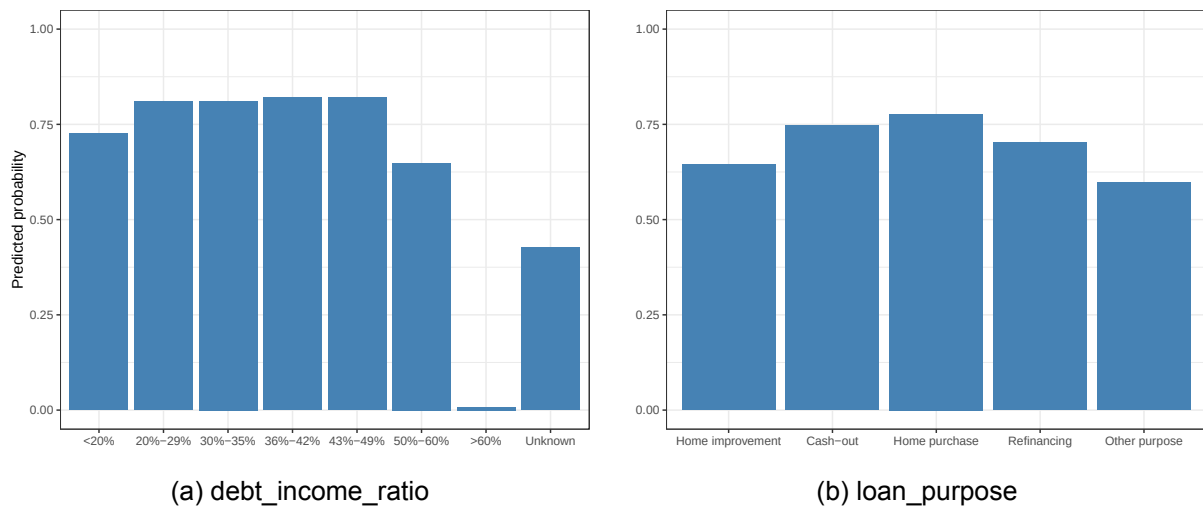


Figure 9: Feature effects for the linear SVM.

Figure 9a shows the effect of the "debt_income_ratio." The probability initially increases slightly and then remains constant at approximately 81% across several categories. Once the ratio reaches the 50%–60% range, the probability drops significantly to about 65%. If the ratio is even higher (>60%), the probability drops to nearly 0%. In cases where the ratio is unknown, the probability is around 43%. Thus, this also shows the strong influence of "debt_income_ratio," consistent with the feature importance results.

Figure 9b displays the effect of "loan_purpose." The "Home purchase" category has the highest approval probability (~78%), followed closely by the "Cash-out" and "Refinancing" categories, with probabilities of about 75% and 70%, respectively. The remaining two categories show considerably lower approval probabilities, at approximately 62%. Consistent with the importance values according to LOCO and LOCI, this feature also has a strong influence.

Finally, we examine the non-linear SVM for our HMDA dataset. First, hyperparameter tuning was performed for the parameters "cost," "gamma," "degree," "coef0," and "kernel." To this end,

random search with 500 evaluations was used. Each evaluation was trained on 66.7% of the data using a holdout split and evaluated on the remaining data. The optimal hyperparameter configuration was found to be "cost = 0.00017," "gamma = 0.005," "coef0 = 1," and "kernel = sigmoid" (the parameter "degree" is not required for the sigmoid kernel).

First, we look at the overall performance of the non-linear SVM with the optimal hyperparameters (see Table 6). The evaluation metrics were calculated using subsampling with a total of 50 repetitions and a training data share of 75%.

Accuracy	Precision	Recall	Specificity	F1-score	log loss
0.833	0.849	0.944	0.51	0.894	0.395

Table 6: Evaluation results for the non-linear SVM.

Similar to the linear SVM, the non-linear SVM achieves an accuracy of approximately 83%, indicating that most observations were correctly classified. The precision is also similar, at about 85%. This indicates that approximately 15% of the predicted approved loans were incorrectly classified.

We also obtain similar values of approximately 94% and 51% for recall and specificity, respectively. Thus, virtually all loans that were actually approved were correctly classified. However, approximately half of the loans that were actually rejected were misclassified by the model.

Next, we analyze the feature importance values of the non-linear SVM (see Figure 10). These values were determined on a test set comprising 25% of the data, using log loss as metric and averaging over 50 repetitions. For training the model, the optimal hyperparameters were used.

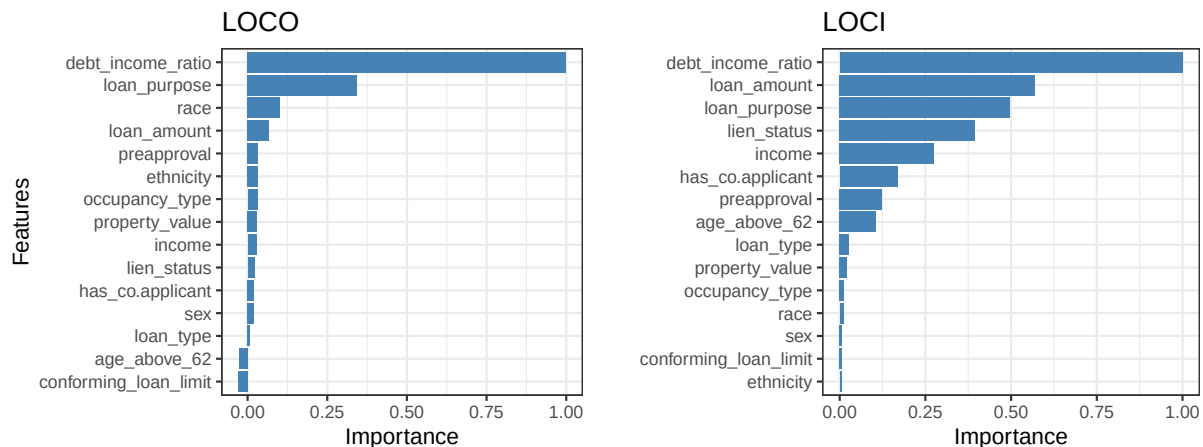


Figure 10: Feature importance values for the non-linear SVM; values are relative to the most important feature.

The results are very similar to those of linear SVM. Both methods identify "debt_income_ratio" as the most important feature. Furthermore, both methods consider "loan_purpose" and "loan_amount" to be significant.

In contrast to LOCI, LOCO classifies the feature "race" as important. Conversely, LOCI identifies the features "lien_status," "income," "has_co.applicant," "preapproval," and "age_above_62" as relevant, whereas LOCO does not.

Finally, we proceed to examine PDPs for a total of four features (see Figure 11). To calculate the PDPs, an additional new model with the optimal hyperparameters was trained on 75% of the data, while the remaining 25% of the data was used to determine the PDPs. The PDPs show the predicted probability of loan approval.

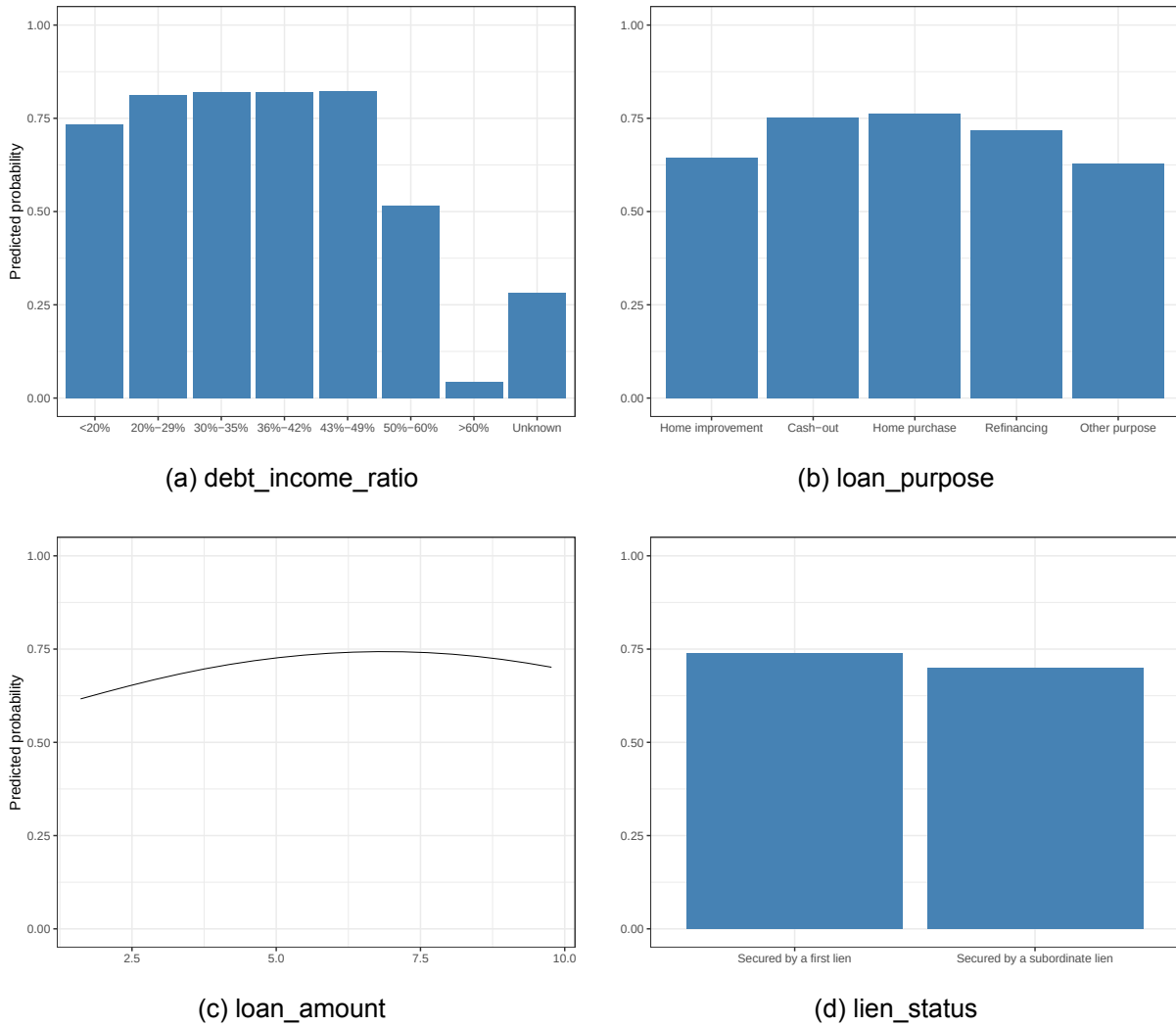


Figure 11: Feature effects for the non-linear SVM.

Figure 11a shows the effect of the "debt_income_ratio." After initially increasing slightly, the probability remains constant at approximately 82% across several categories. Beyond this point, the probability drops rapidly. In the 50%–60% range, the probability is only around 52%, and in the >60% range, it is about 4%. In unknown cases, the probability is approximately 28%. Thus, a strong influence of "debt_income_ratio" can be observed, which aligns with the results of feature importance.

Figure 11b presents the effect of "loan_purpose." The "Home purchase" and "Cash-out" categories have the highest approval probabilities, at approximately 75%. They are followed closely by the "Refinancing" category. The last two categories have significantly lower approval probabilities of about 63%. This feature also demonstrates a relatively strong influence on the model.

Figure 11c displays the effect of "loan_amount." As illustrated, the effect takes the form of a parabola. The probability starts at approximately 61% and increases almost linearly, reaching a peak at about 74%. After that, the probability decreases again to about 70%.

Figure 11d shows the effect of the feature "lien_status." There is no significant difference between the two categories. The approval probability for the "Secured by first lien" category is approximately 74%, which is only slightly higher than that for the "Secured by subordinate lien" category, at about 70%.

7 Interpretable vs. Non-Interpretable Models

In the following sections, we compare the previously discussed models, with a particular focus on the comparison between the interpretable and non-interpretable models.

First, we look at the performance of the models (see Section 7.1). Next, we compare the results of the feature importance methods (see Section 7.2). Finally, we analyze the PDPs for several selected features across the models (see Section 7.3).

7.1 Performance

In this section, we evaluate the performance of the different models. The results of the evaluation metrics for each model are summarized in Table 7.

Model	Accuracy	Precision	Recall	Specificity	F1-score	log loss
Logistic Rgression	0.838	0.854	0.944	0.532	0.897	0.37
Classifiaction Tree	0.842	0.858	0.944	0.545	0.899	0.384
Random Forest	0.848	0.862	0.948	0.559	0.903	0.359
Linear SVM	0.833	0.848	0.945	0.507	0.894	0.396
Non-Linear SVM	0.833	0.849	0.944	0.51	0.894	0.395

Table 7: Comparison of evaluation results for all models.

When comparing interpretable and non-interpretable models, an interesting point becomes apparent: There is little to no discernible difference between the two types of models. The values of the evaluation metrics are nearly identical for all models. This outcome is somewhat counter-intuitive, as one might expect non-interpretable models to demonstrate superior performance in comparison to interpretable models.

In detail, among the interpretable models, the classification tree performs as well as or better than logistic regression across all metrics, except for the log loss, where logistic regression achieves a better value. This indicates that logistic regression is slightly more confident in its predictions than the classification tree.

Among the non-interpretable models, the random forest proves to be the best model, achieving the best values across all metrics. Moreover, the random forest even outperforms all other models, both interpretable and non-interpretable.

We can also observe little to no difference in performance between the linear and the non-linear SVMs. For our dataset, the more complex non-linear SVM does not offer any improvement, so a linear SVM is perfectly adequate. It is also noteworthy that both SVMs have considerably lower specificity values than all other models, although the specificity is generally low across all models.

In summary, the results show that interpretable models can compete with non-interpretable models. Although the non-interpretable random forest achieves the best results, both interpretable models deliver comparable values and differ only minimally in the evaluation metrics. In cases where interpretability is an important factor, interpretable models offer an attractive alternative without sacrificing much performance.

7.2 Feature Importance

In this section, we compare the results of the feature importance methods across the different models. For this comparison, only the top six features, as determined by LOCO and LOCI, are considered. The results are summarized in Figure 12. The LOCO results are shown in blue, and the LOCI results in orange.

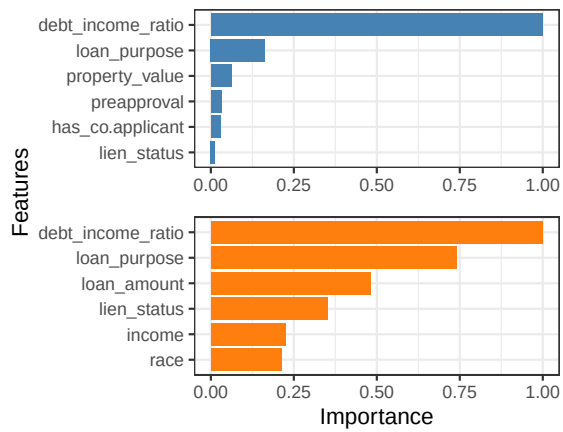
There is no notable difference between LOCO and LOCI when comparing interpretable and non-interpretable models overall. However, noteworthy differences between the models emerge for two specific features. The feature "income" appears in all models, but in the interpretable models it is among the top six features only according to LOCI. In the non-interpretable models (except for non-linear SVM) it is among the top six features only according to LOCO. The feature "property_value" appears among the top six features only in the two interpretable models and in the random forest according to LOCO (and also in the classification tree according to LOCI). In the other two non-interpretable models, this feature does not appear among the top six according to either LOCO or LOCI.

In general, the results are similar across all models, with hardly any major differences. Without exception, "debt_income_ratio" is identified as the most important feature by both LOCO and LOCI in every model. The results for "loan_purpose" and "loan_amount" are also very consistent across the models. With the exception of the linear and non-linear SVM, "loan_purpose" is consistently the second most important feature according to both LOCO and LOCI. In both SVMs, however, it ranks third according to LOCI. Conversely, according to LOCI, "loan_amount" is the third most important feature in all models, except for the SVMs, where it is the second most important feature.

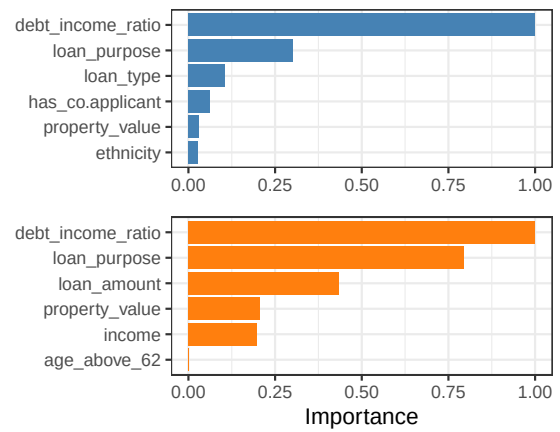
Consequently, this shows that the features "debt_income_ratio" and "loan_purpose" contain highly relevant information about the subject matter and are indispensable for all models. The feature "loan_amount" also carries substantial information for the task. However, since it is important only according to LOCI, this indicates that other features also capture this information.

Despite the general similarity of the results, there are also notable differences between the models. The feature "lien_status" does not appear among the top six in the classification tree; however, it is among the top six features in all other models. In logistic regression, this feature ranks among the top six according to both LOCO and LOCI, while in the other models, it is included among the top six only according to LOCI. There are also differences regarding the two features, "preapproval" and "loan_type." The "preapproval" feature is among the six most important features in logistic regression and the two SVMs according to LOCO. It does not appear among the top six in the other models. A similar situation exists for "loan_type," which appears among the top six only according to LOCO in the classification tree and random forest, but not in the other models according to either LOCO or LOCI.

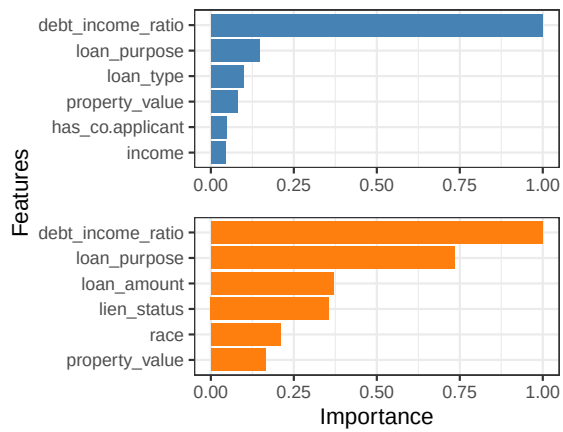
Finally, we examine the "race" and "ethnicity" features to analyze the fairness of the models. The feature "race" appears among the top six features in all models, except the classification tree, whereas "ethnicity" appears only in the classification tree and the non-linear SVM. However, both features have relatively low importance values and, therefore, play only a minor role compared to other features.



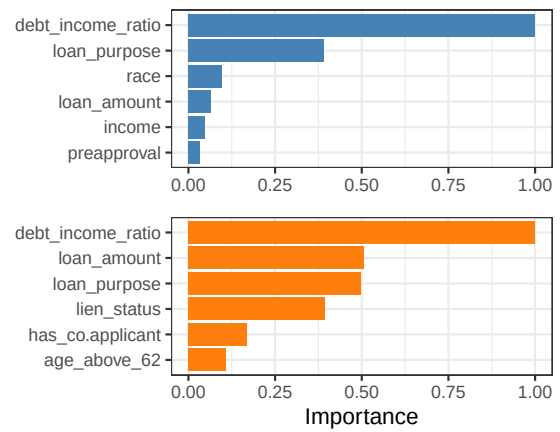
(a) Logistic Regression



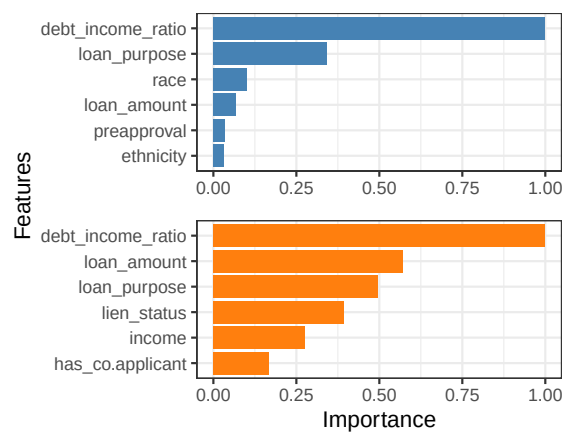
(b) Classification Tree



(c) Random Forest



(d) Linear SVM



(e) Non-Linear SVM

Figure 12: Six most important features for each model.

7.3 Feature Effect

Next, we compare the PDPs of the different models for several selected features. Figure 13 displays the corresponding PDPs. As in the previous sections, the PDPs show the predicted probability of loan approval.

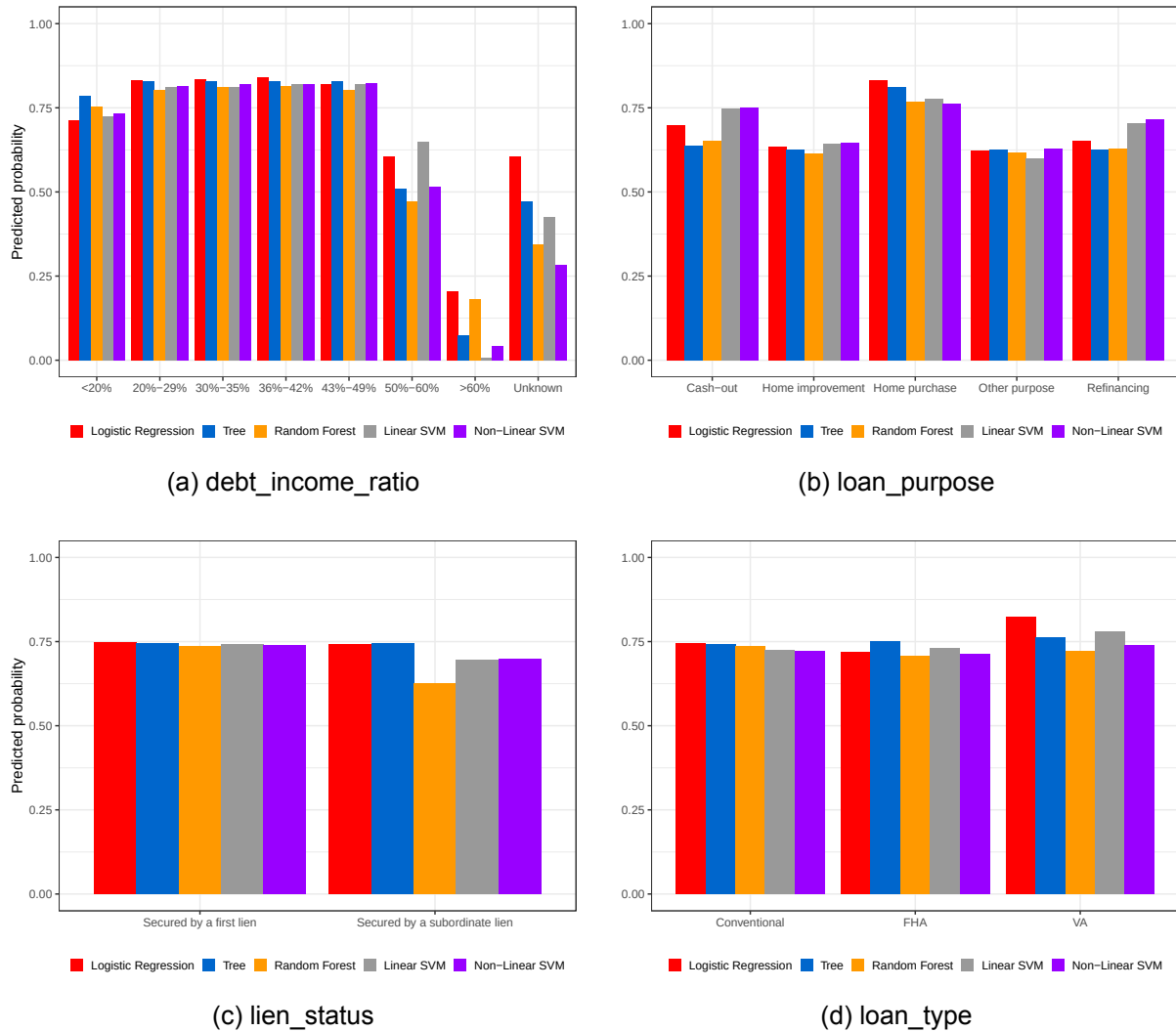


Figure 13: Feature effect comparison for selected features.

Figure 13a illustrates the differences in the effects of "debt_income_ratio." It is readily apparent that the probability pattern across all categories is similar for all models. Initially, the predicted probability increases slightly for all models and then remains nearly constant across several categories. The classification tree is especially notable, as it predicts a substantially higher probability at the beginning (<20%) compared to the other models. After that, the values drop sharply in the 50%–60% and >60% ranges for all models. In the 50%–60% range, logistic regression and the linear SVM stand out due to their notably higher predicted probabilities. Clear differences between the models are also evident in the >60% category. Both logistic regression and the random forest yield estimates that are substantially higher than those of the other models. The linear SVM is also noteworthy in this category, as it is the only model that

predicts a probability very close to zero. Furthermore, the models produce markedly different results in the "Unknown" category, with predicted values differing considerably between the models. Notably, the logistic regression model stands out by predicting a much higher value compared to the other models. It is also interesting that the two interpretable models predict the highest probabilities in this category. Overall, the models show smaller differences in the first categories, while the discrepancies become more pronounced in the later categories.

Next, we examine the effects of the feature "loan_purpose" in Figure 13b. Here, as well, the probability pattern is consistent across all models. All models predict the highest probabilities for the "Home purchase" category. However, a clear distinction emerges between interpretable and non-interpretable models in this category. The two interpretable models demonstrate higher values compared to the three non-interpretable models. Within each model type, the values are highly comparable, i.e., the interpretable models are similar to each other, as are the non-interpretable models. In the "Cash-out" category, the logistic regression, the linear SVM, and the non-linear SVM stand out due to their higher predicted values. The linear and non-linear SVMs also predict higher probabilities than the other models in the "Refinancing" category. In the last two categories, the effects are very similar across all models. Overall, the differences between the models are only apparent in certain categories.

Now, we look at the effects of the feature "lien_status" in Figure 13c. There are no major differences in the first category, "Secured by first lien," as the predicted probabilities are approximately equal across all models. In the second category, "Secured by subordinate lien," greater differences emerge. Here, the interpretable models yield comparable values but generally predict higher probabilities than the non-interpretable models. Among the non-interpretable models, both the linear and non-linear SVMs stand out by predicting higher probabilities than the random forest. Thus, for this feature, there is only one notable difference between the models, which occurs in the second category.

In Figure 13d, we proceed to compare the effects of the feature "loan_type." In the "Conventional" category, only minor differences are observed between the models. Greater discrepancies are evident in the "FHA" category, where the classification tree and the linear SVM predict slightly higher probabilities, while the other models yield approximately identical values. The largest differences are seen in the "VA" category. Logistic regression, the classification tree, and the linear SVM all have higher values than the other two models, with logistic regression having the highest value by a large margin.

Finally, we analyze the features "race" and "ethnicity" in Figure 14 to gain more insight into the fairness of the models.

Figure 14a shows the different effects of "race." In general, all models predict the highest probability of loan approval for White applicants and the lowest probability for Black or African American applicants. Logistic regression stands out by assigning the highest value for White applicants, whereas the classification tree predicts the highest probability for Black or African American applicants. For Asian applicants, the values fall between those of these two groups, with the interpretable models generally predicting higher values. Interestingly, the classification tree assigns the same probability to all categories, indicating that this feature is not used in that model.

Figure 14b shows the effects of the feature "ethnicity." All the models predict the highest probabilities for the "Not Hispanic or Latino" category and the lowest for the "Hispanic or Latino" category. In general, however, the differences between the models are quite small. For the "Not Hispanic or Latino" and "Unknown" categories, the values are very similar across all models. For

the "Hispanic or Latino" category, on the other hand, the classification tree and random forest models produce slightly higher probabilities.

Overall, it is evident that both features influence the prediction of credit approval. Therefore, it can be concluded that the models are not entirely fair with respect to the features "race" and "ethnicity."

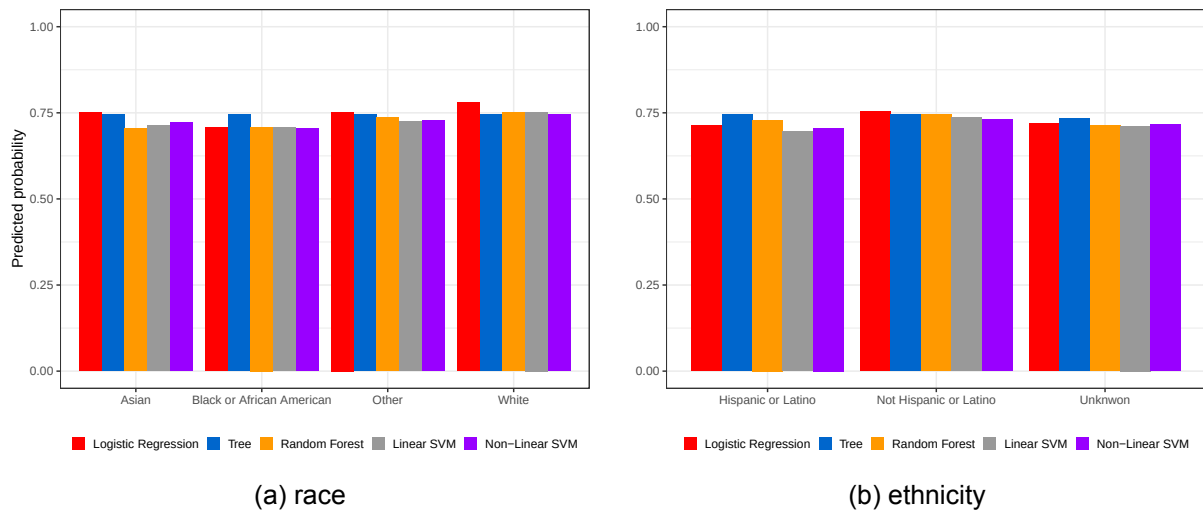


Figure 14: Feature effect comparison for fairness features.

8 Rashomon Effect

In machine learning, the Rashomon effect refers to the phenomenon whereby many diverse models can achieve nearly identical performance on the same problem, yet arrive at their predictions in very different ways. These models may have fundamentally different internal structures, leading to significant variation in aspects such as feature importance. In other words, each model has found a distinct way to solve the problem. A set of such models is also referred to as a Rashomon set. (see Breiman, 2001b)

The Rashomon effect is a critical factor for model interpretability. In some cases, multiple models with comparable performance may yield divergent feature importance values or different PDPs for a given feature. As a result, the models can produce partially contradictory results. This makes interpretation less clear and makes it more challenging to draw conclusions about the data. (see Molnar et al., 2022)

In Section 7.1, we compared the performance of our various models and found that the evaluation metrics differ only slightly, with all five models performing similarly. In Sections 7.2 and 7.3, we examined the feature importances and selected PDPs for the models, identifying minor differences between them. For instance, while the features "debt_income_ratio" and "loan_purpose" consistently show high feature importance values across all models, the PDPs for these two features can differ notably. We also observed differences in the feature importance results for features such as "preapproval" and "loan_type."

Overall, the Rashomon effect is apparent in our results.

9 Discussion

In this paper, we have examined interpretable and non-interpretable machine learning models—logistic regression, classification tree, random forest, linear SVM, and non-linear SVM—for predicting credit approval based on an HMDA dataset. Our research yielded several key findings. First, it is noteworthy that all models achieve nearly identical performance values (Section 7.1). Contrary to widely held assumptions, the more complex, non-interpretable models do not outperform the interpretable models. This outcome aligns with the findings from studies conducted in other domains (see e.g., Chakraborty et al., 2021, Fung et al., 2021). This result lends support to the hypothesis that interpretable models are a viable alternative to non-interpretable models in the context of predicting credit approval.

However, it is also apparent that all models have the same difficulty in correctly classifying rejected loans (see Section 7.1), as evidenced by their low specificity values. The potential causes of this problem could be the limitations of the features or the strong imbalance of the dataset. With additional features or a training dataset with a higher proportion of rejected loans, the models could recognize better patterns among the rejected loans. This issue is particularly critical in our context because an incorrect approval of a loan poses a significant risk to credit institutions. Consequently, future work should focus on improving specificity. In contrast, all models achieve high recall values, meaning they are highly effective at correctly classifying approved loans.

From an interpretability standpoint, the feature importance methods LOCO and LOCI consistently identify the features "debt_income_ratio" and "loan_purpose" as the most important features across all models, irrespective of whether the model is interpretable or non-interpretable (Section 7.2). This finding indicates that these two features contain essential information for accurately predicting loan approval. However, discrepancies in the feature importance results and the PDPs are observed (see Section 7.3). These deviations reflect the so-called Rashomon effect (Section 8), which complicates drawing clear conclusions about the true structure of the data.

Finally, we address some limitations of the study and outline potential future research directions. It is important to note that the dataset used for our analysis exclusively encompasses loan applications from a single year (2023) and a specific region (the District of Columbia). This may limit the generalization of our results, as lending behavior could vary across regions and over different time periods. Since the District of Columbia is a highly urban region, it is conceivable that our results can differ greatly from those that would be obtained in more rural regions. Future work could examine and compare results from urban and rural regions in greater detail. A comparison with data from other urban regions (e.g., New York) would also be an interesting avenue for future research.

10 Conclusion

In this study, various interpretable and non-interpretable models were trained to predict credit approval. These models were then analyzed and compared in terms of their performance and interpretability.

The findings revealed that interpretable models achieve performance levels very similar to those of non-interpretable models. The random forest model demonstrated the highest performance; however, the differences in performance between this model and the others were minimal. Notably, all models exhibited consistently poor specificity, indicating challenges in correctly classifying rejected loan applications.

In general, the models showed only minor differences in their feature importance results and their PDPs. In particular, the features "debt_income_ratio" and "loan_purpose" emerged as the most important features across all models.

In summary, this bachelor's thesis demonstrates that interpretable models can compete with non-interpretable models without sacrificing performance. Interpretable models represent a viable alternative, especially in situations where model transparency and accountability are essential.

A Appendix

The following Tables 8 and 9 explain all features used for modeling (see *Public HMDA - LAR Data Fields | HMDA Documentation*, n.d.).

Name	Description	Possible Values
action_taken	The action taken on the covered loan	- Loan approved - Loan denied
age_above_62	Whether the applicant age is 62 or above	- Yes - No
conforming_loan_limit	Whether the reported loan amount exceeds the government sponsored enterprise conforming loan limit	- C (Conforming) - NC (Nonconforming) - Unknown
debt_income_ratio	The ratio of the applicant's total monthly debt to the total monthly income	- <20% - 20%-29% - 30%-35% - 36%-42% - 43%-49% - 50%-60% - >60% - Unknown
ethnicity	Ethnicity of the applicant	- Hispanic or Latino - Not Hispanic or Latino - Unknown
has_co.applicant	Whether there was a co-applicant	- Yes - No
income	Log transformed gross annual income (in thousands of dollars) of the applicant	- Varying numeric values
lien_status	Lien status of the property securing the covered loan	- Secured by first lien - Secured by subordinate lien
loan_amount	Log transformed amount of the covered loan (in thousand of dollars)	- Varying numeric values
loan_purpose	The purpose of covered loan	- Cash-out - Home improvement - Home purchase - Refinancing - Other purpose
loan_type	The type of covered loan	- Conventional - FHA - VA

Table 8: Feature explanation.

Name	Description	Possible Values
occupancy_type	Occupancy type for the dwelling of the applicant	- Principal residence - Second residence - Investment property
preapproval	Whether the covered loan involved a request for a preapproval	- Preapproval requested - Preapproval not requested
property	Log transformed value of the property securing the covered loan (in thousand of dollars)	- Varying numeric values
race	Race of the applicant	- Asian - Black - White - Other
sex	Sex of the applicant	- Male - Female - Unknown

Table 9: Feature explanation.

B Electronic appendix

Data, code and figures are provided in electronic form.

GitHub repository:

<https://github.com/daniel-mueller17/Interpretable-ML-for-Credit-Approval>

C Informations on used AI Tools

ChatGPT: <https://chatgpt.com>

Used for ideas for writing the introduction and the conclusion

DeepL: <https://www.deepl.com>

Used for translation and writing assistance

References

- Bishop, C. M. (2006). *Pattern recognition and machine learning*, Information science and statistics, Springer, New York.
- Boser, B. E., Guyon, I. M. and Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers, *Proceedings of the fifth annual workshop on Computational learning theory*, ACM, Pittsburgh Pennsylvania USA, pp. 144–152.
URL: <https://dl.acm.org/doi/10.1145/130385.130401>
- Breiman, L. (2001a). Random Forests, *Machine Learning* **45**(1): 5–32.
URL: <https://doi.org/10.1023/A:1010933404324>
- Breiman, L. (2001b). Statistical Modeling: The Two Cultures (with comments and a rejoinder by the author), *Statistical Science* **16**(3): 199–231. Publisher: Institute of Mathematical Statistics.
URL: <https://projecteuclid.org/journals/statistical-science/volume-16/issue-3/Statistical-Modeling-The-Two-Cultures-with-comments-and-a/10.1214/ss/1009213726.full>
- Breiman, L., Friedman, J., Olshen, R. A. and Stone, C. J. (1984). *Classification and Regression Trees* (1st ed.), Chapman and Hall/CRC, New York.
- Chakraborty, D., Başağaoğlu, H. and Winterle, J. (2021). Interpretable vs. noninterpretable machine learning models for data-driven hydro-climatological process modeling, *Expert Systems with Applications* **170**: 114498.
URL: <https://linkinghub.elsevier.com/retrieve/pii/S0957417420311428>
- Chen, C.-C. and Li, S.-T. (2014). Credit rating with a monotonicity-constrained support vector machine model, *Expert Systems with Applications* **41**(16): 7235–7247.
URL: <https://linkinghub.elsevier.com/retrieve/pii/S0957417414003157>
- Cortes, C. and Vapnik, V. (1995). Support-vector networks, *Machine Learning* **20**(3): 273–297.
URL: <https://doi.org/10.1007/BF00994018>
- Doshi-Velez, F. and Kim, B. (2017). Towards A Rigorous Science of Interpretable Machine Learning, *arXiv: Machine Learning* .
URL: <https://www.semanticscholar.org/paper/Towards-A-Rigorous-Science-of-Interpretable-Machine-Doshi-Velez-Kim/5c39e37022661f81f79e481240ed9b175dec6513>
- Ewald, F. K., Bothmann, L., Wright, M. N., Bischl, B., Casalicchio, G. and König, G. (2024). A Guide to Feature Importance Methods for Scientific Inference, in L. Longo, S. Lapuschkin and C. Seifert (eds), *Explainable Artificial Intelligence*, Springer Nature Switzerland, Cham, pp. 440–464.
- Fahrmeir, L., Kneib, T., Lang, S. and Marx, B. (2013). *Regression: Models, Methods and Applications*, Springer, Berlin, Heidelberg.
URL: <https://link.springer.com/10.1007/978-3-642-34333-9>
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine, *The Annals of Statistics* **29**(5): 1189–1232. Publisher: Institute of Mathematical Statistics.
URL: <https://www.jstor.org/stable/2699986>

- Fung, P. L., Zaidan, M. A., Timonen, H., Niemi, J. V., Kousa, A., Kuula, J., Luoma, K., Tarkoma, S., Petäjä, T., Kulmala, M. and Hussein, T. (2021). Evaluation of white-box versus black-box machine learning models in estimating ambient black carbon concentration, *Journal of Aerosol Science* **152**: 105694.
URL: <https://www.sciencedirect.com/science/article/pii/S0021850220301798>
- Guotai, C., Abedin, M. Z. and E-moula, F. (2017). Modeling credit approval data with neural networks: an experimental investigation and optimization, *Journal of Business Economics and Management* **18**(2): 224–240. Number: 2.
URL: <https://journals.vilniustech.lt/index.php/JBEM/article/view/1181>
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning*, Springer Series in Statistics, Springer, New York, NY.
URL: <http://link.springer.com/10.1007/978-0-387-84858-7>
- HMDA Data Browser (n.d.). [Online; accessed 20-May-2025].
URL: https://ffiec.cfpb.gov/data-browser/data/2023?category=states&items=DC&actions_taken=1,3&getDetails=1
- Hodges, H., Garrity, C. and Pope, J. (2024). Deep Learning, Feature Selection, and Model Bias with Home Mortgage Loan Classification: International Conference on Pattern Recognition Applications and Methods, *Proceedings of the 13th International Conference on Pattern Recognition Applications and Methods* **1**: 248–255. Publisher: SciTePress.
- Home Mortgage Disclosure Act (HMDA) | FFIEC (n.d.). [Online; accessed 20-May-2025].
URL: <https://www.ffiec.gov/data/hmda>
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021). *An Introduction to Statistical Learning: with Applications in R*, Springer Texts in Statistics, Springer US, New York, NY.
URL: <https://link.springer.com/10.1007/978-1-0716-1418-1>
- Jung, A. (2022). *Machine Learning: The Basics*, Machine Learning: Foundations, Methodologies, and Applications, Springer Nature, Singapore. ISSN: 2730-9908, 2730-9916.
URL: <https://link.springer.com/10.1007/978-981-16-8193-6>
- Lang, M., Binder, M., Richter, J., Schratz, P., Pfisterer, F., Coors, S., Au, Q., Casalicchio, G., Kotthoff, L. and Bischl, B. (2019). mlr3: A modern object-oriented machine learning framework in R, *Journal of Open Source Software* **4**(44): 1903.
URL: <https://joss.theoj.org/papers/10.21105/joss.01903>
- Leal, A. A. (2024). Algorithms, Creditworthiness, and Lending Decisions, in D. Moura Vicente, R. Soares Pereira and A. Alves Leal (eds), *Legal Aspects of Autonomous Systems*, Springer International Publishing, Cham, pp. 321–353.
- Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J. and Wasserman, L. (2018). Distribution-Free Predictive Inference for Regression, *Journal of the American Statistical Association* **113**(523): 1094–1111. Publisher: ASA Website.
URL: <https://www.tandfonline.com/doi/full/10.1080/01621459.2017.1307116>
- Loyola-González, O. (2019). Black-Box vs. White-Box: Understanding Their Advantages and Weaknesses From a Practical Point of View, *IEEE Access* **7**: 154096–154113.
URL: <https://ieeexplore.ieee.org/abstract/document/8882211>

- Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences, *Artificial Intelligence* **267**: 1–38.
URL: <https://linkinghub.elsevier.com/retrieve/pii/S0004370218305988>
- Molnar, C. (2025). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 3 edn, Leanpub.
URL: <https://christophm.github.io/interpretable-ml-book>
- Molnar, C., Casalicchio, G. and Bischl, B. (2018). iml: An R package for Interpretable Machine Learning, *Journal of Open Source Software* **3**(26): 786.
URL: <https://joss.theoj.org/papers/10.21105/joss.00786>
- Molnar, C., König, G., Herbringer, J., Freiesleben, T., Dandl, S., Scholbeck, C. A., Casalicchio, G., Grosse-Wentrup, M. and Bischl, B. (2022). General Pitfalls of Model-Agnostic Interpretation Methods for Machine Learning Models, in A. Holzinger, R. Goebel, R. Fong, T. Moon, K.-R. Müller and W. Samek (eds), *xxAI - Beyond Explainable AI: International Workshop, Held in Conjunction with ICML 2020, July 18, 2020, Vienna, Austria, Revised and Extended Papers*, Springer International Publishing, Cham, pp. 39–68.
URL: https://doi.org/10.1007/978-3-031-04083-2_4
- Pillai, S. G., Woodbury, J., Dikshit, N., Leider, A. and Tappert, C. C. (2020). Machine Learning Analysis of Mortgage Credit Risk, in K. Arai, R. Bhatia and S. Kapoor (eds), *Proceedings of the Future Technologies Conference (FTC) 2019*, Vol. 1069, Springer International Publishing, Cham, pp. 107–123. Series Title: Advances in Intelligent Systems and Computing.
URL: http://link.springer.com/10.1007/978-3-030-32520-6_10
- Public HMDA - LAR Data Fields | HMDA Documentation (n.d.). [Online; accessed 20-May-2025].
URL: <https://ffiec.cfbp.gov/documentation/publications/loan-level-datasets/lar-data-fields>
- Tharwat, A. (2021). Classification assessment methods, *Applied Computing and Informatics* **17**(1): 168–192. Publisher: Emerald Publishing Limited.
URL: <https://doi.org/10.1016/j.aci.2018.08.003>
- Verdinelli, I. and Wasserman, L. (2024). Feature Importance: A Closer Look at Shapley Values and LOCO, *Statistical Science* **39**(4): 623–636. Publisher: Institute of Mathematical Statistics.
URL: <https://projecteuclid.org/journals/statistical-science/volume-39/issue-4/Feature-Importance-A-Closer-Look-at-Shapley-Values-and/10.1214/24-STS937.full>
- Weng, C.-H., and Huang, C.-K. (2021). A Hybrid Machine Learning Model for Credit Approval, *Applied Artificial Intelligence* **35**(15): 1439–1465. Publisher: Taylor & Francis _eprint: <https://doi.org/10.1080/08839514.2021.1982475>.
URL: <https://doi.org/10.1080/08839514.2021.1982475>
- Zhang, Y. (2022). Machine Learning, Alternative Data, and Mortgage Bias.
URL: <https://papers.ssrn.com/abstract=4055828>
- Zhao, X., Zhang, L., Zhu, G., Cheng, C., He, J., Traore, S. and Singh, V. P. (2023). Exploring interpretable and non-interpretable machine learning models for estimating winter wheat evapotranspiration using particle swarm optimization with limited climatic data, *Computers and Electronics in Agriculture* **212**: 108140.
URL: <https://linkinghub.elsevier.com/retrieve/pii/S0168169923005288>

Declaration of authorship

I hereby declare that the report submitted is my own unaided work. All direct or indirect sources used are acknowledged as references. I am aware that the Thesis in digital form can be examined for the use of unauthorized aid and in order to determine whether the report as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future Theses submitted. Further rights of reproduction and usage, however, are not granted here. This paper was not previously presented to another examination board and has not been published.

Munich, July 20, 2025

Daniel Müller